Software Requirements Specification for Keystone: Capstone Project Management System

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

November 14, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 1
- 1.2 Scope . . . 1
- 1.3 Stakeholders . . . 3
- 1.4 Definitions and Acronyms . . . 4
- 1.5 Characteristics of Intended Reader . . . 6

2 System Overview **6**
- 2.1 Problem Statement . . . 6
- 2.2 Product Vision . . . 8
- 2.3 System Context . . . 9
- 2.4 User Characteristics . . . 12
- 2.5 System Constraints . . . 12

3 Stakeholder Requirements **13**

4 User Personas **15**
- 4.1 Persona 1: Priya, Capstone Student . . . 15
- 4.2 Persona 2: Marcus, External Supervisor . . . 16
- 4.3 Persona 3: Dr. Malik, Internal Supervisor . . . 16
- 4.4 Persona 4: Dr. Reyes, Course Instructor . . . 17

5 Use Cases **17**
- 5.1 UC_SubmitProposal: Submit a Project Proposal . . . 17
- 5.2 UC_ReviewProposal: Review and Approve a Proposal . . . 18
- 5.3 UC_BrowseProjects: Browse Available Projects . . . 19
- 5.4 UC_SelectProject: Team applying for a project . . . 20
- 5.5 UC_ProposeOwnProject: Propose a Student-Initiated Project . . . 21
- 5.6 UC_TeamProfile: Create and Maintain a Team Profile . . . 21
- 5.7 UC_ViewTeam: View Team Details and Contacts . . . 22
- 5.8 UC_ValidateTeams: Validate and Export Team Composition . . . 23
- 5.9 UC_FollowUpUnselected: Identify Unselected Projects . . . 23
- 5.10 UC_ViewTimelines: View Course Timelines . . . 24
- 5.11 UC_NotifyChecklist: Receive Notifications and Complete Tasks . . . 24
- 5.12 UC_ApproveTeamProject: Submit and Approve a Team's Project . . . 25
- 5.13 UC_ImportEnrollment: Import Course Enrollment . . . 26

6 Functional Requirements **27**

7 Data Requirements **37**
- 7.1 Major Data Entities . . . 37
- 7.2 Project and Team Status Lifecycle . . . 39

7.2.1 Entity Relationships . . . 39
7.3 Data Dictionary . . . 41
7.4 Privacy and Data Retention . . . 48
7.4.1 Storage . . . 49
7.4.2 Data Retention and Deletion . . . 49
7.4.3 Privacy Controls . . . 49
7.4.4 Consent . . . 51
7.4.5 Encryption . . . 51
7.4.6 Compliance . . . 51
7.4.7 Enrollment Data . . . 52

8 Non-Functional Requirements 52
8.1 Look and Feel Requirements . . . 52
8.1.1 Appearance Requirements . . . 52
8.1.2 Style Requirements . . . 52
8.2 Usability and Humanity Requirements . . . 52
8.2.1 Ease of Use Requirements . . . 52
8.2.2 Personalization and Internationalization Requirements . . . 53
8.2.3 Learning Requirements . . . 53
8.2.4 Understandability and Politeness Requirements . . . 53
8.2.5 Accessibility Requirements . . . 53
8.3 Performance Requirements . . . 53
8.3.1 Speed and Latency Requirements . . . 53
8.3.2 Safety-Critical Requirements . . . 54
8.3.3 Precision or Accuracy Requirements . . . 54
8.3.4 Robustness or Fault-Tolerance Requirements . . . 54
8.3.5 Capacity Requirements . . . 54
8.3.6 Scalability or Extensibility Requirements . . . 55
8.3.7 Longevity Requirements . . . 55
8.4 Operational and Environmental Requirements . . . 55
8.4.1 Expected Physical Environment . . . 55
8.4.2 Wider Environment Requirements . . . 55
8.4.3 Requirements for Interfacing with Adjacent Systems . . . 55
8.4.4 Productization Requirements . . . 55
8.4.5 Release Requirements . . . 56
8.5 Maintainability and Support Requirements . . . 56
8.5.1 Maintenance Requirements . . . 56
8.5.2 Supportability Requirements . . . 56
8.5.3 Adaptability Requirements . . . 56
8.6 Security Requirements . . . 56
8.6.1 Access Requirements . . . 56
8.6.2 Integrity Requirements . . . 57
8.6.3 Privacy Requirements . . . 57

8.6.4 Audit Requirements . . . 58
8.6.5 Immunity Requirements . . . 58
8.7 Cultural Requirements . . . 58
8.7.1 Cultural Requirements . . . 58
8.8 Compliance Requirements . . . 58
8.8.1 Legal Requirements . . . 58
8.8.2 Standards Compliance Requirements . . . 58

9 Requirements Traceability Matrices **59**

10 Rationale **62**
10.1 Scope . . . 62
10.2 Functional Requirements . . . 62
10.3 Non-Functional Requirements . . . 62
10.4 Data Requirements . . . 63
10.5 AI Requirements . . . 64

11 Likely Changes **64**

12 Unlikely Changes **66**

13 Appendices **67**
13.1 Appendix A: Additional Materials . . . 67
13.1.1 Stakeholder Interview Notes . . . 67
13.2 Appendix B: Development Plan . . . 73

14 Development Plan **73**
14.1 Phase 1: Core Workflow (implemented in the course) . . . 74
14.2 Phase 2: Communication and Visibility (implemented in the course) . . . 74
14.3 Phase 3: Extended Features (implemented if time allows) . . . 74
14.4 Future Work (not implemented, or faked for the demo) . . . 75
14.5 Faked or Simplified for the Course . . . 75
14.6 Appendix C: Values of Auxiliary Constants . . . 75
14.7 Appendix D: Generative AI Disclosure . . . 77
14.8 Appendix E: Reflection . . . 77

List of Figures

1 System Context . . . 10
2 Project and team status lifecycle . . . 41

List of Tables

1 Definitions and Acronyms 6
2 Entity Relationships 40
3 Data Dictionary 41
4 Visibility by record type and state 50
5 What an external supervisor can read 51
6 Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements 60
7 Traceability Between Functional and Nonfunctional Requirements 62
8 Stakeholder interviews 67
9 Auxiliary Constants 77

Revision History

Date	Version	Notes
Oct 13th 2026	1.0	Introduction and System Overview
Oct 18th 2026	1.1	Use Cases and User Personas
Oct 18th 2026	1.2	Data Requirements
Oct 19th 2026	1.3	Non-functional Requirements
Oct 28th 2026	1.4	Add more requirements after peer review and interviews
Oct 29th 2026	1.5	NFR re-organization, additions
Oct 30th 2026	1.5	Addressed conflicts in defintions and acronyms, added likely changes, and added more requirements after peer review and interviews
Nov 13th 2026	1.6	Updated Likely Changes section to match the Module Guide review

1 Introduction

This software requirements specification (SRS) document describes the system and product vision as well as the functional and non-functional requirements for Keystone. The SRS is intended to be a reference for the development team, stakeholders, and future maintainers of the system. This document is constructed using the IEEE 29148 and IEEE 15288 standards as a guide (ISO/IEC/IEEE, 2018, 2023).

1.1 Purpose

The purpose of Keystone is to make capstone project proposals, discovery, team formation, and team project approval transparent, consistent, and easy to manage for the students, supervisors, and course instructors involved.

Capstone project and team selection currently happens across disconnected tools such as email, online forms, discussion forums, and shared files, leading to a lack of visibility, clarity, and detail in information available to students, supervisors, and instructors. As a result, project descriptions do not set clear expectations, students and supervisors cannot easily tell which projects are available or learn about each other, and instructors spend significant time manually coordinating the process. See Problem Statement for a more detailed description of the problem.

Keystone is intended to solve these problems by giving every participant a single, trustworthy source of information about capstone projects, teams, and their status, from the moment a project is proposed until it is approved.

1.2 Scope

Problem being solved. The fragmented and unreliable flow of information in capstone project and team selection and approval, as described in the Problem Statement section below.

Intended users. Capstone students, project supervisors (McMaster faculty and industry sponsors), course instructors, and teaching assistants (TAs), across the Software Engineering (SE), Computer Science (CS), and Mechatronics (TRON) capstone courses.

Major functionality. Keystone covers the lifecycle from project proposal to team project approval. Within that boundary the system:

- provides proposal templates for supervisors and for students, so that projects are described consistently;
- lets instructors review supervisors' project proposals before they are listed, and agree changes with the supervisors;

- provides a searchable catalog of projects showing which are open, taken, or able to take more teams;
- lets students and teams create structured profiles of their skills, experience, interests, and work style that other students and supervisors can view, indicate whether they are looking for a team, and find teammates interested in the same projects;
- suggests compatible teammates and suitable projects to students, based on their profiles and preferences
- lets student teams express interest in supervisor-proposed projects;
- supports communication between all users regarding project proposals, team formations, applications and and approvals, ensuring discussions are organized and accessible to relevant parties;
- supports comment threads on projects, student posts, and applications, so that questions and answers are kept in one place;
- lets supervisors and student teams each accept or decline working together, and records when both have agreed;
- notifies users of events that need their attention, such as new projects, status changes, and replies;
- provides supervisors with course process, timeline, and expectation information, and examples of past capstone projects, configurable by the course instructor;
- gives instructors visibility across courses into projects, teams, and their status, including which supervisors' projects were not selected, so that those supervisors can be informed;
- provides guidance and automation to help students meet all requirements when submitting their projects for approval, and when submitting changes to their project information;
- lets instructors and TAs approve, request changes to, or reject a team's project submission and later changes to its information;
- lets instructors and TAs populate a course with its enrolled students from Avenue to Learn (or by file upload), and shows which enrolled students are not yet on a team;
- lets instructors configure which features are enabled and disabled for full flexibility

What Keystone does not do.

- It does not judge whether a project is feasible, sufficiently complex, or appropriate for a capstone course; this remains the responsibility of the course instructor.

- It does not make approval decisions. Final approval of teams and projects always rests with the course instructor and TAs. Courses may carry out approval inside the platform or in an external process.
- It does not resolve intellectual property (IP), confidentiality, or non-disclosure questions between students and supervisors.
- It does not automatically match supervisors to student-proposed projects. Teams may name a supervisor on their proposal or contact one themselves, and supervisors can browse the list of student-proposed projects, but Keystone does not recommend, assign, or maintain an opt-in list of supervisors for them.
- It does not schedule meetings or provide direct messaging between users. Meetings take place outside the platform, and discussion is limited to comment threads on posts and applications.
- It does not handle supervision after a project begins, such as feedback requests to supervisors, team performance concerns, or grading.

Expected inputs. Inputs are text-based project and profile information entered by users, course enrollment lists, course information, and project status changes made by authorized users.

Expected benefits. Students can make more informed decisions on which projects to pursue, and find compatible teammates; supervisors know who is interested in their projects, learn about the students, and receive fewer repeated emails; and instructors spend less time on repetitive coordination and gain visibility into the whole process.

1.3 Stakeholders

- **Customer:** Course instructors for the SE, CS, and TRON capstone courses (Dr. Smith, Dr. Moradi, and Dr. Wassyng respectively). They decide which features their course uses and make the final project and team approval decisions.
- **End Users:**
  - Capstone students, who browse projects, create profiles, form teams, and apply to projects.
  - Project supervisors, who propose projects, manage their capacity, and review and confirm teams. Supervisors are either McMaster faculty or industry sponsors.
  - Course instructors, who configure their course, populate it with enrolled students, approve project proposals, approve team projects, and manage users and project status.
  - TAs, who help instructors administer the course. TAs can populate enrolled students and course information, view projects and teams, and participate in the

team project approval process. TAs do not take part in the approval of project proposals. Instructors and TAs may also hold the administrator permission, which lets them manage users' course association and roles, read the external-provider error log, and delete data that other users cannot.

- **Development Team:** Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib.

1.4 Definitions and Acronyms

Term	Definition
Capstone course	A final-year design course in which student teams spend roughly eight months (September to April) designing, implementing, and validating a system for a real group of stakeholders. In this document, the SE, CS, and TRON capstone courses.
SE, CS, TRON	Software Engineering, Computer Science, and Mechatronics Engineering, the three programs in the Computing and Software Department whose capstone courses Keystone supports.
Course instructor	The instructor responsible for a capstone course and for approving its teams and projects.
Supervisor	A McMaster faculty member or industry sponsor who proposes a project, or who agrees to advise a team on a project, and provides domain knowledge or acts as a client.
Industry sponsor	A supervisor from outside McMaster, who may have no McMaster account (MacID).
Project proposal	A description of a project submitted by a supervisor, or a student, for teams to consider.
Student-proposed project	A project proposed by a student team rather than by a supervisor.
Project status	The position of a project proposal or listing in its lifecycle, for example Draft, Open, Full, or Closed. See Figure 2.
Team-project status	The position of one team's relationship to a project, for example Applied, Agreed, or Approved. Each team on a project has its own. See Figure 2.
Student profile	A student's skills, experience, interests, work style, and project preferences, visible to other students and to supervisors.
Team profile	A description of a team's members and shared goals, project interests, and work style.

Supervisor capacity	The number of teams a supervisor is willing to supervise on a project.
Team formation rules	Course rules on team composition: which programs may work together on which projects, and the minimum and maximum team size.
Proposal approval	The course instructor's review of a supervisor's project proposal before it is listed for students. The instructor may list it as submitted, work with the supervisor to make changes, or reject it.
Team project approval	The course instructor or TA's decision to approve a team's selected project and membership
Approval mode	How a course carries out team project approval: in-platform (review runs in Keystone), external (review happens outside Keystone; an instructor or TA records the outcome manually), or integrated (an approval adapter reads the outcome from the external system).
Approval adapter	A component that reads the approval outcome of a team project from an external approval system and updates the team-project status. Integrated mode is only available for courses that have an adapter.
Custom document	A deliverable, chosen by each team from a list, that suits the type of project (for example, a usability testing report).
TA	Teaching assistant who supports a capstone course. TAs can populate course information, view projects and teams, and take part in team project approval, and may be assigned to specific teams. A TA does not take part in project proposal approval.
Administrator	An instructor or TA with the administrator permission (`is_admin` set), who can manage users' course association and roles, read the external-provider error log, and delete data that other users cannot. Not a separate role.
MacID	A McMaster University user account identifier.
Avenue to Learn (Avenue)	McMaster's learning management system. It holds course enrollment for courses that have an Avenue site, which is optional.
Enrollment list	The list of students enrolled in a capstone course, imported from Avenue to Learn or uploaded from a file.
External supervisor	A supervisor who signs in without a McMaster account. Every external sign-in is an external supervisor, who is *unverified* until an instructor approves a project proposal that the supervisor submitted, and *verified* afterwards.

Verified external supervisor	An external supervisor whose project proposal has been approved by an instructor. Can additionally read student profiles and student-proposed projects. Never able to read other supervisors' proposals or projects.

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

The intended reader of this SRS should have a background in software engineering, including knowledge of software development processes, requirements engineering, and system design principles. They do not have to be experts on these topics, but they should at least have the knowledge of a final year undergraduate student in software engineering. No knowledge of the capstone course process beyond the definitions in this section is required.

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

2.1 Problem Statement

Current situation. Capstone team and project selection is spread across disconnected tools, including email, online forms, MS Teams, Avenue to Learn and GitLab. Supervisors submit proposals through a form. The SE course instructor, who collects proposals for all three courses, reviews them, discusses any changes with the supervisor by email, and edits the list by hand before it is distributed to students as a static list. Students then form teams informally, mainly through discussion forums on Avenue to Learn / Teams and personal contacts, contact supervisors by email, and submit their chosen project and team for instructor approval through a course-specific process. No single place shows the state of the whole process.

Problems. The following problems were identified through interviews with stakeholders (see Stakeholder Interview Notes) and general observation.

- **Project descriptions are inconsistent and do not set expectations.** Proposals vary widely in the level of detail, and very short descriptions put students off. Some of the things students want to know include a project's end goal and expected impact. Also, descriptions often do not state how much of the project already exists. Supervisors are unsure what a capstone project should look like and what a proposal should include, and students who pitch their own ideas do not know what a proposal needs. This makes projects difficult to compare and leads to mismatched expectations.

- **Student-proposed projects are hard to see.** Students' own project ideas appear only in discussion forums, where supervisors cannot see them, including supervisors who would like to supervise a student-proposed project. They are not tracked alongside supervisor-proposed projects.
- **There is no visibility of project status.** Students and supervisors cannot see which projects are taken or open, or whether a supervisor is willing to take more than one team, and after a rejection students do not know which projects remain. New projects and updates are hard to notice, and users do not want to have to remember to check for them.
- **Compatible teammates are hard to find.** Students without an existing network rely on forum posts or chance. Teammates' project preferences, skills, and work styles are often unknown until a team has formed, so teams can disagree about what to build.
- **Supervisors do not know the students' backgrounds.** Supervisors have little structured information about the students and teams who contact them, such as their skills, experience, and program. They can only gain this information through interviews or rely on outside sources such as LinkedIn profiles, which are inconsistent and not always available.
- **Communication during selection is inefficient.** Supervisors receive repeated emails from teams, including after a team has been chosen. What one team learns about a project in a meeting is not shared with other teams applying for the same project. Nobody can see which teams are in contact with which supervisors. There is also no record that a team and a supervisor agreed to work together. Students sometimes report that a supervisor agreed when the supervisor has no record of the team, so the instructor must confirm agreements separately.
- **Supervisors lack guidance before proposing and responding.** Before proposing a project, supervisors receive only a short summary of the course's expectations and process. The full course information arrives only after teams and projects are settled, and supervisors want it earlier, along with examples of past projects. Supervisors are also unsure how quickly to reply to students and whether to speak to every team that contacts them. This leads to misunderstandings about what a capstone project should deliver and to many questions to the instructor.
- **Instructors lack visibility across courses.** Instructors cannot see which projects students in other capstone courses selected, so they cannot tell which proposed projects were not selected. Supervisors whose projects are not selected are not informed, which is a possible risk to future project proposals.
- **Instructor review is repetitive and time consuming.** Instructors review each team's submission manually and often give the same comments to many teams on simple items, such as missing information, wasting the instructor's time. When reviewing project proposals, changes to proposals that are agreed with supervisors by email must

then be applied to the project list by hand.

- **Records drift after approval.** Team and project information changes after approval (for example, titles, supervisors, links, team composition, and TA assignments), but updating the official record takes extra steps, so it is often not updated and the instructor's records become out of date.
- **Course enrollment is assembled by hand.** Instructors compare the list of enrolled students against team lists by hand to find students who are not yet on a team, and getting enrolled students into a new platform or a team workspace is not trivial. Enrollment data lives in Avenue to Learn, which not every course uses.

Impact. Students lose time early in a project of limited duration, teams end up mismatched with their projects or with each other, and course instructors and TAs carry the administrative burden of the process. A possible further risk is that supervisors become reluctant to propose projects in later years if their projects go unselected or their expectations are not met.

Desired improvement. All participants share a single, accurate and current source of information about projects, supervisors' capacity, teams, and approval status; students can find suitable projects and teammates; supervisors can see students' skills and ideas and receive information about the course when they need it; and course instructors keep full control of approval decisions while spending less time on repetitive tasks.

2.2 Product Vision

Keystone will be the central place where capstone projects are proposed, discovered, and matched with student teams, and where instructors can see and manage the process. Each course decides which features it uses and how its approval process is carried out, so the platform can be adopted by different courses, departments, and universities.

The goals of Keystone are:

1. **Consistent project information.** Every project, whether proposed by a supervisor or by students, is described in a consistent format that makes its objectives, expected impact, required skills, current state, and expected outcomes clear.
2. **Visibility of availability and updates.** Users can tell which projects are open, taken, or able to take additional teams, and are made aware of new projects and changes without having to check for them.
3. **Easier team formation.** Students can describe their skills, experience, work style, and project preferences in a structured profile, find teammates with compatible profiles and preferences, and propose projects of their own for others to join.
4. **Informed matching.** Students receive suggestions of compatible teammates and suitable projects based on their profiles and preferences.

5. **Informed supervisors.** Supervisors can see student-proposed projects and the skills and experience of the students and teams interested in their projects.
6. **Efficient communication during selection.** Questions and answers about a project or application are kept in one place, supervisors can see which teams are interested in their projects, and fewer repeated emails are needed.
7. **Clear agreement between supervisors and teams.** A supervisor's and a team's agreement to work together is recorded in the platform and is visible to the instructor.
8. **Timely guidance for supervisors.** Supervisors have access to information about the capstone course, its timeline, and its expectations, and to examples of past projects, when they consider proposing a project, as configured by the course instructor.
9. **Oversight for instructors.** Instructors can see projects, teams, supervisors, and their status across courses, including projects that were not selected.
10. **Instructor-controlled approval with less repetitive work.** Final approval decisions remain with the course instructor. The platform supports the project proposal approval process, guiding supervisors to write more detailed project proposals and facilitating discussion between the instructor and supervisor. The platform supports the team project approval process carried out in the course, whether that takes place in the platform or elsewhere, helps students submit complete and correct information, and helps keep records current.
11. **Configurable and reusable.** Features can be turned on or off, and course-specific rules can be configured, so that the platform serves different courses without modification.

2.3 System Context

The entities that interact with Keystone are students, supervisors, course instructors, TAs, authentication providers, an email service, Avenue to Learn, and the external approval systems of courses that do not carry out approval within Keystone.

- Student Responsibilities:
  - Provide an accurate profile including skills, experience, interests, work style, and project preferences.
  - Provide accurate team information and project proposals, including student-proposed projects.
  - Express interest in projects, take part in discussions, and respond to supervisors' decisions.
  - Submit complete project and team information for approval and keep it current.
- Supervisor Responsibilities:

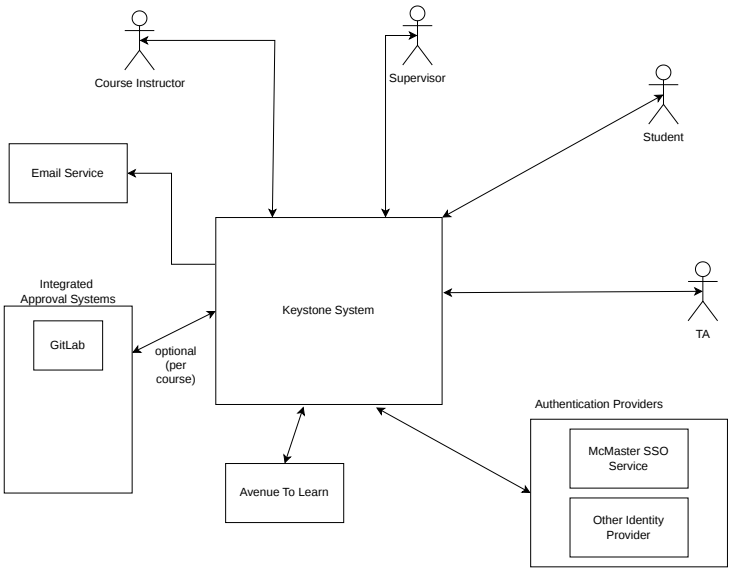


Figure 1: System Context

  - Provide accurate, complete project proposals, including the current state of the project, expected outcomes, and the number of teams they are willing to supervise.
  - Review interested teams, take part in discussions, and accept or decline working with each team.
  - Keep their capacity and project information current.

- Course Instructor Responsibilities:
  - Populate the platform with course information, including the enrollment list, course process, timeline, expectation information and examples of past projects.
  - Configure the features, rules, and approval mode that apply to their course.
  - Make final decisions on teams and projects, including project proposal and team project approvals, requests for changes, and rejection.
  - Judge project feasibility and appropriateness for the course.
  - With the administrator permission: manage users' course association and assigned roles, review the log of external-provider errors, and carry out deletions that ordinary users cannot make.

- TA Responsibilities:
  - Populate the platform with course information, including the enrollment list, course process, timeline, expectation information and examples of past projects.

  - Assist the instructor in team project approvals, requests for changes, and rejections.
  - Update project and team status as directed by instructors, in courses where approval takes place outside Keystone.
  - Assist instructors in keeping records accurate.
  - With the administrator permission: manage users' course association and assigned roles, review the log of external-provider errors, and carry out deletions that ordinary users cannot make.
- Authentication Provider Responsibilities:
  - Verify the identity of users who sign in, including users with and without McMaster accounts.
- Email Service Responsibilities:
  - Deliver the notifications generated by Keystone to users' email addresses.
- External Approval System Responsibilities:
  - In courses that carry out approval outside Keystone, carry out approval of teams and projects. In integrated mode, make the outcome available to an approval adapter, which updates Keystone. In external mode, instructors or TAs record the outcome in Keystone manually.
- Avenue to Learn Responsibilities:
  - For courses that have an Avenue site, provide the list of enrolled students (name, email or MacID, program, and course) to Keystone.
- Keystone Responsibilities:
  - Provide proposal templates, and check that required project, profile, and team information is present and well formed.
  - Show current project status, availability, and supervisor capacity.
  - Apply the team formation rules configured for each course.
  - Keep discussions attached to the project, post, or application they concern, and record whether a supervisor and a team have agreed to work together.
  - Import and check course enrollment lists, show a preview before applying them, and show which enrolled students are not on a team.
  - Restrict what each user can see and change according to their role.
  - Record who made each change to a project or team, and when.

  - Notify users of events that need their attention.
  - Provide instructors with an overview of projects, teams, and their status across courses.
  - Suggest compatible teammates and projects based on profile data.

Keystone is used to coordinate an academic process. Use is concentrated around the start of the academic year, when projects are proposed and teams are formed, with lighter use afterwards. It is not mission-critical or safety-critical. A failure delays the coordination of projects and teams but does not cause harm. The data handled includes personal information about students, so privacy is the main risk to be managed, rather than safety.

2.4 User Characteristics

- **Students** are final-year undergraduate students in the SE, CS and TRON programs. They are comfortable using web applications and should be able to figure out how to use Keystone without any training. They use Keystone mainly at the start of the course. They are motivated to find a suitable project quickly, but differ in whether they already have a team or a project idea.
- **Supervisors** are McMaster faculty or industry sponsors. They have a wide range of backgrounds. They may use Keystone only a few times a year, have limited time, and may be unfamiliar with the capstone course, its expectations, and its timeline. Industry sponsors may have no McMaster account. No technical training beyond using a web application can be assumed, and the effort needed to propose a project must stay low enough that it does not discourage them.
- **Course instructors** know the course's process and rules. They configure the platform for their course and make approval decisions. They use it throughout the term, and need to configure it without developer help.
- **TAs** are graduate students who know the course's process. They help instructors with day-to-day administration and are comfortable using web applications and technical tools. Instructors and TAs with the administrator permission also manage users and roles, review the error log, and handle data deletion requests, and need no technical knowledge beyond using the web interface.

2.5 System Constraints

- **Web-based.** Keystone shall be accessible through current versions of common web browsers on desktop, tablet, and mobile devices, and shall not require specialized hardware or installation. An internet connection is required.
- **Modular features.** Each major feature shall be able to be enabled or disabled independently for each course, so that a course can use only the parts that fit its process.

- **Independent of any one approval tool.** Keystone shall not depend on a specific external tool, such as the GitLab merge request process used by the Software Engineering course, to function. It shall support courses that carry out approval in the platform (in-platform mode), courses that carry it out elsewhere and record the outcome manually (external mode), and, where an approval adapter exists, courses that synchronize with an external process (integrated mode).
- **Users outside the university.** Keystone shall support users who do not have a McMaster account (such as industry sponsors) alongside users who do. An external identity provider cannot confirm who a person is, so external supervisors have restricted access until an instructor approves one of their project proposals (see `FR_ExternalSupervisorVerifica` and `FR_ExternalSupervisorScope`).
- **Configurable course rules.** As Keystone should be extensible to different capstone courses, course-specific rules, such as which programs may work together on which projects and the minimum and maximum team size, shall be set by the course instructor as configuration and shall not be fixed in the design.
- **Configurable enrollment source.** Keystone shall obtain a course's enrollment list from Avenue to Learn where the course has an Avenue site, and shall also accept an uploaded file, since having an Avenue site is optional.

3 Stakeholder Requirements

`SR_ProposalTemplate` Supervisors and instructors shall be able to propose a project using an structured template rather than a unstructured description. The template should include the expected scope/goals, the capstone team's deliverables, project impact, industry or field of study, confidentiality, and IP concerns. (S, SEI, ES)

`SR_OptionalFields` The proposal template shall clearly distinguish required fields from optional ones, so that standardization does not discourage potential supervisors from submitting good projects and ideas. (SEI)

`SR_ProposalGuidance` Supervisors and student teams shall have access to guidelines, an FAQ section, and examples of past capstone projects to help them generate ideas and judge appropriate size, scope, and content. (S, ES)

`SR_CourseInfoEarly` The course outline, calendar, objectives, and content-coverage expectations shall be available to potential supervisors before they propose a project. (ES)

`SR_StudentProposals` Student teams shall have a documented process for proposing their own project, including a template and guidance on how to calibrate the scope. Student-proposed projects shall be visible to supervisors so that teams and supervisors can find each other without the system matching them. (S, IS, CI)

`SR_BrowseProjects` Students can browse through all available projects in one place and see which projects are availible versus already assigned to a team. (S)

`SR_ProjectStatus` All stakeholders should be able to see the status of all projects they are permitted to view (e.g., open, full, closed) and the stage each team has reached on a project (e.g., applied, agreed, approved) at any time. (S, ES, SEI, CI)

`SR_StatusNotify` Stakeholders shall be notified when the status of a project they care about changes via email. (S, ES)

`SR_UnselectedProjects` The instructors and TAs will be able to identify which projects have not been selected after project selection, in order to follow up with their supervisors. (SEI)

`SR_HumanApproval` Approval decisions shall be made by the instructor and TAs and not automated. Routine checks and repeated feedback should be automated to reduce their workload. (SEI)

`SR_ReviewDialogue` Students and the instructor shall be able to exchange feedback iteratively during the approval process, preserving student exposure to review-based workflows. (SEI, S)

`SR_StudentProfiles` Students should be able to provide background information (team bio, interests, skills, experience, linkedln link) that supervisors can see before meeting a team. (ES, IS)

`SR_TeamValidity` The instructor and TAs can get accurate, up-to-date, and validated team composition data (e.g., all students are in exactly one team). (SEI)

`SR_TeamSubmissionChecks` When a team submits its project for approval, the system will check that all required information, documents, and repository setup exist. (SEI)

`SR_CrossCourseTimelines` Supervisors working with teams across multiple capstone courses shall be able to see how the timelines and requirements differ between those courses. (IS)

`SR_TeamContacts` Supervisors can see each team's members, team number, the point of contact, and assigned TA. (ES, IS)

`SR_PushNotify` Stakeholders shall be notified of tasks and deadlines through channels they already use (outlook), so that they do not have to remember to check for them. (ES)

`SR_Checklist` Each stakeholder shall have a checklist of the actions required of them and their current completion state. (ES)

`SR_SinglePlace` Stakeholders should be able to do capstone coordination (projects, schedule, teams, communication, and notes) in one place. (S, ES)

`SR_ReduceAdmin` Administrative tasks including the generation of a list of approved projects, groups, selected projects, and supervisors will be automated. (SEI, ES)

`SR_TAApproval` TAs shall be able to approve team projects in the same way as instructors, but shall not take part in the approval of project proposals. (SEI)

`SR_CourseConfig` Instructors shall be able to choose which features their course uses and set their course's rules, so that a course with a simpler process is not burdened by features it does not need. (SEI, TI, CI)

`SR_Enrollment` Instructors and TAs shall be able to populate a course with its enrolled students without entering each student by hand, using Avenue to Learn or through file upload, and they shall be able to see which enrolled students are not yet on a team. (SEI, TI)

`SR_ExternalSupervisorAccess` External supervisors shall be able to sign in without a McMaster account and immediately see the course information, past project examples, and proposal template, so that they can propose a project. Their confidential project ideas shall not be visible to other external supervisors or to people with no connection to the course. Once an instructor approves an external supervisor's proposal, they shall also see student profiles and student-proposed projects, but never other supervisors' projects. (CI, SEI, ES)

4 User Personas

4.1 Persona 1: Priya, Capstone Student

- Name: Priya Nair
- Description: Software engineering level IV student. She formed a team of friends over the summer and had a few project ideas before the term started.
- Goals: Land a project that matches her team's interests and skills. Also know what is expected from the supervisor, due-dates, and if the team is on track. Keep communication with the supervisor coherent.
- Frustrations: Vague one-paragraph project descriptions (sometimes even a few sentences), not knowing whether a project is still open or already taken, and needing many open software applications to find deadlines, meeting details, and contacts. She worries about ending up with no project before the deadline for project selection.
- Technical Ability: Ability is medium. She is comfortable with GitLab, JSON, and web tools, but still wants the process to be intuitive as it will reduce stress.
- Based on: S `S_Vague`, S `S_Uncertainty`, S `S_Status`, S `S_ManyTabs`, S `S_OnePlace`

4.2 Persona 2: Marcus, External Supervisor

- Name: Marcus Tremblay
- Description: Senior engineer at a local company who proposed a capstone project for the first time. He has a full-time job and supervises the team in his spare time. He has no university login and is not familiar with course details.
- Goals: Submit a well-scoped proposal with minimal effort. Understand what the course expects of a team much earlier in the process. Learn about his team's skills before committing (not just meeting them, maybe have access to their linkedin profiles). Be reminded of tasks via email rather than having to remember to check a platform everytime.
- Frustrations: Open-ended proposal boxes that make expectations unclear, no information about students before meeting them, a lot of back and forth between with course instructor to get course information and searching through email CC lists to find who to reply to.
- Technical Ability: Ability is medium to high. He is comfortable with new applications, but has little patience for keeping track of multiple platforms.
- Based on: ES `ES_Template`, ES `ES_NoStudentInfo`, ES `ES_ManyLogins`, ES `ES_NotifyNotRemember`, ES `ES_WhoToEmail`

4.3 Persona 3: Dr. Malik, Internal Supervisor

- Name: Dr. Hassan Malik
- Description: Professor who submits proposals yearly and supervises several teams, often across more than one capstone course (SE, CS, Tron). He also helps with presentations and Expo posters when he has time, but is not involved in running the course.
- Goals: Know the schedule and requirements of each course his teams are in. Know who his team members and TAs are without having to search through many platforms. Give end-of-term feedback without a lot of effort.
- Frustrations: The three courses have different timelines that are not well coordinated, and there is no standard procedure for how a group finds a supervisor. He is unsure when project approval happens and often forgets to send reports.
- Technical Ability: Ability is medium. He uses email and standard web tools daily, but wants a simple interface.
- Based on: IS `IS_Schedule`, IS `IS_MultiCourse`, IS `IS_Procedure`, IS `IS_KnowTA`, IS `IS_Reports`

4.4 Persona 4: Dr. Reyes, Course Instructor

- Name: Dr. Daniel Reyes
- Description: Professor who runs the capstone course. Each year he recruits supervisors, collects proposals, approves projects, and records team details. Today this involves several forms, a spreadsheet, scripts, and merge requests.
- Goals: Spend his time on decisions such as project viability and approval, rather than administration. Have accurate, valid team and project data produced automatically. Know which projects were not selected so he can follow up with their supervisors.
- Frustrations: Sending the same feedback to multiple groups, manually maintaining team data, and having to recruit supervisors to have enough projects. He does not want automation that makes approval decisions, and does not want strict standards that discourage good projects.
- Technical Ability: Ability is high. He is comfortable with scripts, GitLab, and CI, and values students being exposed to them.
- Based on: SEI `I_CurrentProcess`, SEI `I_TeamJSON`, SEI `I_Decisions`, SEI `I_Unselected`, SEI `I_Automate`, SEI `I_RepeatFeedback`

5 Use Cases

5.1 UC_SubmitProposal: Submit a Project Proposal

Primary Actor

External Supervisor or Internal Supervisor (Marcus, Dr. Malik)

Preconditions

- The proposal submission period is open.
- The supervisor has access to the system.

Main Success Scenario

1. The supervisor chooses to propose a new project.
2. The system shows the proposal template, with required and optional fields marked, links to guidelines, an FAQ, examples of past projects, and the course outline.
3. The supervisor fills in the project title, organization, contact, scope and goals, expected team deliverables, project impact, field of study, confidentiality, and IP concerns.
4. The supervisor submits the proposal.
5. The system confirms receipt and gives the proposal the status “submitted”.

6. The system notifies the instructor of the new proposal.

Alternative Flows

- 3a. The supervisor leaves a required field empty: the system highlights it and does not allow submission.
- 3b. The supervisor is not ready to finish: the system saves a draft that can be completed later.
- 4a. The submission period is closed: the system informs the supervisor and does not accept the proposal.

Postconditions

- The proposal is stored with the status "submitted" and is visible to the instructor.

Related Requirements: SR_ProposalTemplate, SR_OptionalFields, SR_ProposalGuidance, SR_CourseInfoEarly

5.2 UC_ReviewProposal: Review and Approve a Proposal

Primary Actor

Instructor (Dr. Reyes)

Preconditions

- At least one proposal has been submitted by a supervisor or a student team.

Main Success Scenario

1. The instructor opens the list of proposals awaiting review.
2. The system shows the instructor a proposal with all the filled out fields, and whether the proposer is a verified supervisor.
3. The instructor assesses the proposal against the approval criteria he has determined.
4. The instructor verifies that the proposer is a verified supervisor and not bad actor.
5. The instructor approves the proposal by listing it.
6. The system changes the status from "Proposal submitted" to "Open" and notifies the proposer. If the proposer is an unverified external supervisor, the system also marks them verified and tells them that they can now see student profiles and student-proposed projects.

Alternative Flows

- 4a. The proposal needs changes: the instructor writes feedback and requests revision. The system sets the status to "Proposal changes requested" and notifies the proposer,

who can respond and resubmit; resubmitting sets the status back to "Proposal submitted". The flow returns to step 2.

- 4b. The proposal does not have enough substance and cannot be fixed: the instructor rejects it with a reason, the system sets the status to "Proposal rejected", and notifies the proposer.

Postconditions

- The proposal has the status "Open", "Proposal changes requested", or "Proposal rejected", and the full feedback history is saved.
- If the proposer was an unverified external supervisor, they are now verified if the proposal was approved.

Related Requirements: SR_HumanApproval, SR_ReviewDialogue, SR_StatusNotify, SR_ReduceAdmin

5.3 UC_BrowseProjects: Browse Available Projects

Primary Actor

Student (Priya)

Preconditions

- The student is registered in the course.
- At least one project is listed (status "Open" or "Full").

Main Success Scenario

1. The student opens the project list.
2. The system shows all listed projects with their title, field of study, summary, and status ("Open" or "Full").
3. The student filters the list (e.g., by field of study or status).
4. The student opens a project to read its full description, deliverables, and supervisor.
5. The student marks the project as one of their team's preferences.

Alternative Flows

- 3a. No project matches the filters: the system says so and offers to clear the filters.
- 4a. A project's status changes while the student is viewing it: the system shows the updated status the next time the page is loaded.

Postconditions

- The student's team has a saved list of preferred projects.

Related Requirements: SR_BrowseProjects, SR_ProjectStatus, SR_SinglePlace

5.4 UC_SelectProject: Team applying for a project

Primary Actor

Student (Priya)

Preconditions

- The student is a member of a registered team.
- The project that the student's team is interested in has status "open".

Main Success Scenario

1. The student selects an open project and expresses their their team's interest in that project.
2. The system checks that the project is still open.
3. The system records the team's application with the team-project status "Applied" and updates the interest count so everyone can see there is a group interested in the project. The project stays "Open": an application uses no supervisor slot, so other teams can still apply.
4. The system notifies the project's supervisor and lets them discuss the project with the team.
5. The system shows the student a confirmation with the supervisor's contact.

Alternative Flows

- 2a. The project is not "Open" (it is "Full" or "Closed"): the system tells the student the project is no longer available and shows the team's other preferences and open projects.
- 3a. The supervisor declines, or the team withdraws, at any point before approval: the team's status becomes "Declined or withdrawn" and interested parties are notified. If a slot had been reserved (the supervisor had accepted), it is released and a "Full" project becomes "Open" again.

Postconditions

- The team has an application with the status "Applied" and the project's status is unchanged. The supervisor may then accept ("Supervisor Accepts"), after which the team may accept the working arrangement to become "Agreed". Team acceptance is unavailable before supervisor acceptance; discussion remains available while the application is "Applied". A slot is reserved when the supervisor accepts, and the project becomes "Full" only when all of its slots are reserved.

Related Requirements: SR_ProjectStatus, SR_StatusNotify, SR_BrowseProjects

5.5 UC_ProposeOwnProject: Propose a Student-Initiated Project

Primary Actor

Student (Priya)

Preconditions

- The student's team has a project idea that is not on the project list.

Main Success Scenario

1. The student chooses to propose their own project.
2. The system shows the process for student-initiated projects, the proposal template, and guidance on appropriate scope.
3. The team fills in the template. The team may name a prospective supervisor they have already contacted.
4. The team submits the proposal.
5. The system notifies the instructor and, if one was named, the prospective supervisor.
6. The proposal proceeds through the review process (see UC_ReviewProposal).

Alternative Flows

- 3a. The team has no supervisor: the proposal is published to the list of student-proposed projects, where supervisors can view it and start a comment thread with the team. The system does not recommend supervisors to the team.
- 5a. The named supervisor declines: the team is notified and may name another supervisor or publish the proposal to the student-proposed list.

Postconditions

- The proposal exists with the status "submitted" and is linked to the team.

Related Requirements: SR_StudentProposals, SR_ProposalTemplate, SR_ProposalGuidance

5.6 UC_TeamProfile: Create and Maintain a Team Profile

Primary Actor

Student (Priya)

Preconditions

- The student is a member of a team.

Main Success Scenario

1. The student opens their team profile.
2. The student enters a team bio, and for each member their interests, skills, experience, and (optionally) a LinkedIn link.
3. The student saves the profile.
4. The system makes the profile visible to the team's supervisor and the instructor.

Alternative Flows

- 2a. A member prefers not to share certain information: the field can be left blank, as only the team bio is required.
- 3a. The team has no project yet: the profile is stored and shown to the supervisor when a project is selected.

Postconditions

- The team's profile is saved and visible to authorized stakeholders.

Related Requirements: SR_StudentProfiles

5.7 UC_ViewTeam: View Team Details and Contacts

Primary Actor

External Supervisor or Internal Supervisor (Marcus, Dr. Malik)

Preconditions

- A team has been assigned to the supervisor's project.

Main Success Scenario

1. The supervisor opens the page for their project.
2. The system shows the team number, members, designated point of contact, and assigned TA.
3. The supervisor reads the team profile (skills, interests, experience) before the first meeting.
4. The supervisor contacts the team's point of contact.

Alternative Flows

- 2a. The team has not finished their profile: the system shows the available details and indicates that the rest is missing.
- 2b. No TA has been assigned yet: the system shows "not yet assigned".

Postconditions

- The supervisor knows who the team is and how to contact them.

Related Requirements: SR_TeamContacts, SR_StudentProfiles

5.8 UC_ValidateTeams: Validate and Export Team Composition

Primary Actor

Instructor (Dr. Reyes), Teaching Assistant

Preconditions

- Students have registered teams in the system.

Main Success Scenario

1. The instructor opens the team composition view.
2. The system checks the data (e.g., every student is in exactly one team, every team has a project and a supervisor).
3. The system shows the result and any issues.
4. The instructor exports the validated team composition data.

Alternative Flows

- 3a. The system finds a problem (e.g., a student in more than one team or in none): it lists the affected students and teams, and the export is blocked until the problem is resolved.

Postconditions

- The exported team data is accurate, up to date, and valid.

Related Requirements: SR_TeamValidity, SR_ReduceAdmin

5.9 UC_FollowUpUnselected: Identify Unselected Projects

Primary Actor

Instructor (Dr. Reyes), Teaching Assistant

Preconditions

- The project selection period has ended.

Main Success Scenario

1. The instructor opens the project list and filters for “Closed” projects with no approved team (unselected projects).

2. The system lists those projects with their supervisors and contact details.
3. The instructor selects the projects and sends a message (e.g., a thank-you) to their supervisors.

Alternative Flows

- 2a. All approved projects were selected: the system shows an empty list.

Postconditions

- The instructor knows which projects were not chosen and can follow up with their supervisors.

Related Requirements: SR_UnselectedProjects, SR_ProjectStatus

5.10 UC_ViewTimelines: View Course Timelines

Primary Actor

Internal Supervisor (Dr. Malik), External Supervisor (Marcus)

Preconditions

- The course schedules have been entered by the instructor.

Main Success Scenario

1. The supervisor opens the course schedule.
2. The system shows the key dates for each course the supervisor's teams are in (e.g., approval, SRS, implementation, validation and user testing, Expo, end-of-term feedback).
3. The supervisor compares the timelines and requirements of the courses.

Alternative Flows

- 2a. The dates for a course are not yet decided: the system marks them "to be announced" and notifies the supervisor once they are set.

Postconditions

- The supervisor knows the expectations and deadlines in each course.

Related Requirements: SR_CrossCourseTimelines, SR_CourseInfoEarly

5.11 UC_NotifyChecklist: Receive Notifications and Complete Tasks

Primary Actor

Any stakeholder (Priya, Marcus, Dr. Malik, Dr. Reyes)

Preconditions

- The stakeholder has an action due (e.g., approve a proposal, submit feedback, finish a team profile) or a relevant project status has changed.

Main Success Scenario

1. The system sends the stakeholder an email about the task or status change, with a link to the relevant item.
2. The stakeholder follows the link.
3. The system shows the stakeholder's checklist of required actions and which are complete.
4. The stakeholder completes the task.
5. The system marks it complete on the checklist.

Alternative Flows

- 1a. The email cannot be delivered: the system shows the notification in the stakeholder's checklist and alerts the instructor.
- 4a. The stakeholder does not complete the task by the deadline: the system sends a reminder.

Postconditions

- The task is recorded as complete and the checklist is up to date.

Related Requirements: SR_PushNotify, SR_Checklist, SR_StatusNotify

5.12 UC_ApproveTeamProject: Submit and Approve a Team's Project

Primary Actor

Student (Priya), Instructor (Dr. Reyes), TA

Preconditions

- A team has been formed.
- The team and supervisor have both accepted (team-project status "Agreed"), on a listed project or a student-initiated one.

Main Success Scenario

1. The student opens the team project submission form for their team.

2. The system shows the required items: team members, supervisor, project, required documents, and a valid project repository.
3. The student fills in the form and attaches the required documents.
4. The student submits the team project for approval; the status becomes "Team project submitted".
5. The system runs automated checks (e.g., all required fields and documents are present, the repository exists and is set up correctly, the instructor is added as a collaborator, every member is on exactly one team).
6. The system forwards the submission, with the results of the checks, to the instructor and TA; the status becomes "Team project under review".
7. The instructor or TA review the submission and approves it.
8. The system changes the team's status to "Approved" and notifies the team and supervisor.

Alternative Flows

- 5a. An automated check fails: the system tells the student exactly what is missing or invalid, with standard guidance for fixing it, and does not forward the submission to the instructor until the issues are fixed. This replaces the repetitive comments the instructor writes today.
- 7a. The reviewer finds problems the checks cannot catch (e.g., scope is too small): the reviewer writes feedback and requests revision. The system sets the status to "Team project changes requested" and notifies the team, who can respond and resubmit (status "Team project submitted"). The flow returns to step 5.
- 7b. The reviewer rejects the submission with a reason: the system sets the status to "Team project rejected", releases the team's slot (a "Full" project becomes "Open"), and notifies the team and supervisor.

Postconditions

- The team's status is "Approved", and the data feeds the team composition export (see UC_ValidateTeams). The project's own status ("Open" or "Full") is unchanged.

Related Requirements: SR_HumanApproval, SR_ReviewDialogue, SR_TeamValidity, SR_ReduceAdmin, SR_TeamSubmissionChecks

5.13 UC_ImportEnrollment: Import Course Enrollment

Primary Actor

Instructor (Dr. Reyes), Teaching Assistant

Preconditions

- The course exists in the system and the actor is course staff for it.

Main Success Scenario

1. The actor chooses to import the course's enrollment list from Avenue to Learn.
2. The system retrieves the list of enrolled students.
3. The system shows a preview listing the students to be added, the students already present, and any students on the platform who are no longer in the list.
4. The actor reviews the preview and confirms the import.
5. The system adds the new students to the course and records the time of the import.
6. The system shows the enrolled students who are not yet on a team.

Alternative Flows

- 2a. Avenue is unavailable, the course has no Avenue site, or the retrieval fails: the system leaves the existing enrollment unchanged, tells the actor, and offers file upload.
- 3a. A student on a team is no longer in the list: the system flags the student for the actor's review and does not remove the student from the team.
- 4a. The actor needs to add a student who is not in the list (e.g., a student who joins late): the actor adds the student manually.

Postconditions

- The course's enrollment list is up to date and the time of the last import is recorded.

Related Requirements: `SR_Enrollment`, `SR_TeamValidity`, `SR_ReduceAdmin`

6 Functional Requirements

`FR_SupervisorProposal` The system shall allow supervisors to create and submit project proposals using a structured template including a title, problem statement/background, objectives, skills/requirements, team deliverables, project impact, supervisor capacity details, level of current completion, hardware/non-hardware, industry, information regarding confidentiality and IP concerns, and additional project terms.

Rationale: Standardizing proposals reduces inconsistencies in project descriptions helping sudents understand exactly what the project is about and allowing supervisors to clearly highlight their expections.

Fit Criterion: Supervisors can successfully fill out the defined fields with the appropriate information and submit a proposal for review.

`FR_ProposalOptionalFields` The system shall distinguish required project proposal fields from optional ones preventing submissions if any required fields are not sufficiently filled.

Rationale: Keeping most fields optional would prevent supervisors from being discouraged to propose a project idea if they lack some details regarding the scope of their project. A projec that lacks a detailed scope can still be a good idea.

Fit Criterion: The system prevents stakeholders from submitting projects where mandatory fields are not sufficiently filled but submits a project that has mandatory fields filled even if some optional fields are missing.

`FR_StudentProposal` The system shall allow student teams to create and publish a project proposal in a separate student-proposed project list using a structured template including the title, problem statement/background, and proposed solution/methodology.

Rationale: Students need a dcumented way to present their own project ideas if they choose not to select from the list of ideas from supervisors.

Fit Criterion: A student team can successfully create a project proposal, fill out the defined fields with the above information, and the proposal appears in a student-proposed project list.

`FR_ReviewProposal` The system shall allow instructors to review project proposals submitted by supervisors and approve, request, or provide feedback and request changes. Approving a proposal submitted by an unverified external supervisor verifies that supervisor (`FR_ExternalSupervisorVerification`).

Rationale: Instructors need to verify that the proposed project is sufficient for a Senior-level capstone project and that the details provided are sufficiet for an understanding of the project.

Fit Criterion: A instructor can access a project proposal then accept, reject, or request changes successfully. Rejection or change requests are sent back to the supervisor for revision while accepted projects are included in the student project catalog.

`FR_ViewApprovedProjects` The system shall provide a centralized catalog where all projects approved by the instructor that the user is permitted to view (Table 4) can be found, filtered, and browsed based on details and availability.

Rationale: Students should be able to find projects and compare the varying opportunities with their team's skills and interests.

Fit Criterion: Users can view all the projects that were approved by the professor, search by title or keywords, and filter by the varying project fields. The filter and search options should also only return the relevant projects.

`FR_ViewProjectStatus` The system shall display the current availability status of each

project in the catalog (open or full) and the maximum number of teams it can take. See Figure 2.

Rationale: It is critical for students and supervisors to have clear visibility which projects are still available for selection. This gives students the ability to still request available projects if they have been rejected from others and supervisors to properly advertise their status.

Fit Criterion: Each project in the catalog clearly displays the availability status and the supervisor capacity. Even if labels are not updated in real-time, the core application should respond to user actions appropriately based on status (i.e. taken projects cannot be requested).

`FR_StudentProfile` The system shall allow students to create and update their profiles providing background information including bio, program, interests, project preferences, skills, experience, work style, and LinkedIn that are viewable by students, TAs, instructors, and approved supervisors.

Rationale: Informative profiles allows students to discover other students with similar skills and interests when forming teams. It also allows supervisors to briefly evaluate interested students before interviewing them.

Fit Criterion: Students can create, save, update, and visit their profiles. These profiles should also be viewable by other students and supervisors.

`FR_StudentTeams` The system shall allow students to create and join teams, invite specific students to join, recruit additional members, and assign roles.

Rationale: Students should be able to use the platform to create teams, join other teams, invite the people they know or fill in remaining spaces on their existing team by recruiting new individuals.

Fit Criterion: Students are able to create a team, send invitations to selected students to which those students can accept or deny, specify number of members needed and assign roles including a point-of-contact. Students can join an existing team.

`FR_ValidateTeamRules` The system shall validate team membership based on up-to-date course team formation rules, including minimum and maximum team sizes, permitted program combinations, and duplicate membership. When course rules change, the system shall show the teams that no longer comply and flag them for instructor review without invalidating them.

Rationale: The capstone course requires that certain constraints are met when forming teams and automating this reduces administrative efforts for TAs and Instructors.

Fit Criterion: The system prevents team formations that violate any of the rules mentioned above.

`FR_TeamDiscovery` The system shall allow students who are seeking teammates to indicate their availability and discover other students or teams looking for members.

Rationale: Students who do not have a team or simply want to join a team instead of creating one will have a catalog of other teams and other students seeking others with similar interests.

Fit Criterion: A student can indicate whether they are looking for a team and they can discover other teams/students looking for additional members.

`FR_TeamProfile` The system shall allow student teams to create and update a shared team profile including the member bios, collective skills, project interests, goals, and work style that are viewable by all users within the course.

Rationale: Team profiles help supervisors evaluate the overal team's interests and whether the team's goals and skills align with the project in a consolidated way without having to go through each team member's profile.

Fit Criterion: Authorized team members can create and modify their team profile based on their team members' profiles. Other users can view these profiles.

`FR_TeamRecommend` The system shall recommend other available students/teams and capstone projects based on shared skills, interests, experience, and project preferences.

Rationale: Recommendations help student obtain the desired experience from the capstone course and work on projects that match their interests while pairing with like-minded students.

Fit Criterion: Students can receive teammate and project recommendations after their profile is complete based on highest similarities.

`FR_ProjectInterest` The system shall allow student teams to express interest in a supervisor-proposed project.

Rationale: A centralized application process allows supervisors to identify interested teams and communicate without relying on back-and-forth email chains.

Fit Criterion: A student can submit an application expressing their interest to the project that can be reviewed by the supervisor.

`FR_DeclineTeam` The system shall allow supervisors to accept or decline requests from student teams to work on their proposed projects, and record when both parties agree to the working arrangement. See Figure 2 Decline decisions shall be communicated using respectful message templates with an optional reason, and the system shall not publish rankings of students or teams.

Rationale: This reduces confusion by explicitely recording when both parties have understood each other's terms and have formally agreed to work with each other.

Fit Criterion: The supervisor can accept or decline an application. Only after supervisor acceptance can the team accept the working arrangement, changing its status from Supervisor Accepts to Agreed. Teams may discuss an application or withdraw before accepting.

`FR_ProjectThreads` The system shall allow any authorized users to create and reply to threads associated with their own project proposals, student team formations, and project applications. For TAs and Professors, that will be any projects in their scope.

Rationale: Users can exchange meaningful feedback, ask for clarification during the project selection stage, and communicate to understand project and course needs.

Fit Criterion: An authorized user can create or respond to a thread under a project proposal, project application, or a team formation they are associated with.

`FR_ThreadsNotify` The system shall notify a user by email when there is a new reply in a thread they are part of, a decision on their application, agreement, or submission, or a status change on a project or team they are associated with. Recipients shall be determined when the notification is sent, and the email body shall contain only a summary of the event and a link, excluding confidential project, IP, and profile fields.

Rationale: Notifications keep related stakeholders informed regarding project-related updates and decisions without repeatedly checking the platform.

Fit Criterion: Any changes to a project's status, decisions from either a student/supervisor, and thread updates, the system sends an email identifying the event to all relevant and authorized users. Students also get notifications for new project listing.

`FR_RequestApproval` The system shall allow student teams to submit (or re-submit after modification given instructor feedback) their selected project and team information for instructor approval when in-platform approval is enabled. See Figure 2.

Rationale: A centralized submission allows students to provide all capstone project related details for instructors to approve without relying on external tools.

Fit Criterion: Student teams can submit their project and team information through in-platform approval process that can be reviewed by instructors.

`FR_ApprovalResponse` The system shall allow course instructors and TAs to approve, reject, or request modifications with feedback to submitted team-project applications when in-platform approval is enabled.

Rationale: Instructors and TAs must have the final say on team project approval

Fit Criterion: An instructor or TA can select an approval decision for the submitted team-project and provide feedback to improve project descriptions or identify missing items.

`FR_ExternalApproval` The system shall support recording capstone project and team approvals either manually by instructors or TAs associated with the project or automatically through an adapter. For approvals synchronized through an adapter, the system shall show the time of the last successful synchronization and flag a failed one.

Rationale: Different capstone courses have different approval systems that they would prefer to keep in place so the system must account for this.

Fit Criterion: For courses without integration adapter, either the TA or the instructor can manually update the status of the project as approved. If there is an adapter available, the system will automatically synchronize and update the approval status based on the information recieved from the external system.

`FR_ApprovalPreflightValidate` The system shall validate submissions before sending them to instructors to ensure project information is complete, the project is available, the team does not already have another approved project, and the submission adheres to course-specific requirements.

Rationale: To reduce instructor workload, repeated feadback and routine checks based on course constriaints should be done automatically.

Fit Criterion: The system will prevent project submissions if any of the requirements above are not met, otherwise, the project can be submitted.

`FR_ApprovalRevision` The system shall allow users to submit changes to previously approved team and project information that will still require the instructor to review before confirming. See Figure 2.

Rationale: Details about projects may change later and require updates.

Fit Criterion: Student teams can propose changes to approved project or team record. The orginal approval remains in effect until the instructor accepts the new changes.

`FR_DisableFeatures` The system shall allow course instructors to configure the platform features including team formation, project approvals, and project proposals rules by disabling any undesired features for their course.

Rationale: Different capstone courses have different requirements and workflows, so instructors must be able to configure or customize the system to meet their course's needs.

Fit Criterion: An instructor can successfully enable or disable a supported features and define constraints that match their course's workflow.

`FR_ReferenceMaterial` The system shall allow course instructors and TAs to publish and maintain the course outline, course resources, FAQs, timelines, and past capstone examples for students and supervisors to view as reference.

Rationale: Early access to course expectations reduces confusion about project scope, deliverables, deadlines, and responsibilities for both the student and supervisor.

Fit Criterion: An instructor or TA can add, edit, and publish course resources where students or supervisors and retrieve published resources.

`FR_RequiredActionsList` The system shall provide users with a checklist of required actions and their completion status based on the course requirements.

Rationale: Stakeholders need a concise way to identify outstanding responsiblities and monitor their progress in the capstone approval procedure. This helps them keep on top of tasks and prevents them from missing critical deadlines.

Fit Criterion: Users can view a checklist containing their assigned actions and corresponding completion states. The system updates completion states for actions that are done and verified, such as submitting a project proposal or completing an approval submission.

`FR_ActionRequiredEmail` The system shall send users administrative email notifications for upcoming deadlines and outstanding actions from a user's checklist.

Rationale: Email notifications will help stakeholders complete required tasks and fulfill their capstone responsibilities without regularly checking in the platform.

Fit Criterion: When configured, the system sends an email identifying incomplete or urgent action items and upcoming deadlines.

`FR_ReadCourseInformation` The system shall provide authorized Instructors and TAs with access and detailed information for projects, teams, supervisors, and their statuses across their assigned capstone course(s) or designated capstone teams.

Rationale: Instructors need visbility to identify unselected projects, coordinate with supervisors, and maintain accurate admin details.

Fit Criterion: An authorized instructor or TA can view and filter project and team statuses, monitor approval progress, and access relevant discussions.

`FR_ExportCourseInformation` The system shall provide an automatically generated exportable report to Instructors and TAs providing a list of approved projects, team compositions, project selections, and supervisors.

Rationale: Automated reports reduce manual administrative work for Instructors and TAs if they want to obtain accurate course-wide records through compiled documentation.

Fit Criterion: An authorized instructor or TA can generate and download reports containing selected admin records reflecting approved projects, team compositions, project selections, and supervisor information.

`FR_MacSSO` The system shall allow users to authenticate via McMaster Single Sign-On (SSO) for McMaster students and staff or another trusted authentication system for external supervisors before accessing any other services.

Rationale: This allows the platform to uphold its security when handling sensitive information and that the account is accessed by the intended individual.

Fit Criterion: A user can sign in using an available identify provider and access the functions available to them.

`FR_AuthorizationRBAC` The system shall restrict access to system functionality and information according to the user's role and course permissions. Users whose role cannot be determined shall have no access to course data. Users who sign in through the external provider shall have only the access given in `FR_ExternalSupervisorScope`.

Rationale: Different stakeholders (Students, TAs, Instructors, and Supervisors) have different roles in the platform so access restrictions are critical to prevent unauthorized access when it comes to viewing and modifying content. An example is that teams should only have access to modify only their team's approved project details. An Instructor can modify any project's details.

Fit Criterion: Users are only able to access information and perform actions permitted by their assigned roles and scope.

`FR_UpdateRoles` The system shall allow authorized administrators to manage users' course association and assigned roles.

Rationale: Different capstone courses involve different participants, and access permissions must reflect each users' responsibilities.

Fit Criterion: An authorized administrator can view and update a user's course association and permitted course role.

`FR_ExternalSupervisorVerification` The system shall give every user who signs in through the external authentication provider the external supervisor role in the unverified state, and shall change an external supervisor to verified only when an instructor approves a project proposal that the external supervisor submitted. Approval of any other proposal (including a student proposal that names the external supervisor), a request for changes, or a rejection shall not verify them. A later withdrawal of the approved proposal shall not undo verification, and an instructor may reverse it by changing the user's role (`FR_UpdateRoles`).

Rationale: An external identity provider accepts anyone, so signing in does not show that a person is an industry sponsor. An instructor's review of a real project proposal is the check that the person is a genuine sponsor.

Fit Criterion: A new external sign-in is unverified and cannot read student profiles. After an instructor approves a proposal that the account submitted, it can read them.

Approval of a student proposal naming the account does not change its state.

`FR_ExternalSupervisorScope` The system shall limit an external supervisor to the access in Table 5. An unverified external supervisor can read course reference material (including past project examples), the proposal template, and their own proposals with the threads on them. A verified external supervisor can additionally read student and team profiles, student-proposed projects, and the applications to their own projects. A request for any other supervisor's proposal or project shall be answered as if the record does not exist, and no catalog, search result, count, status, or notification shall reveal it.

Rationale: Industry project ideas are confidential, while student profiles hold information comparable to a public professional profile. Student information should still not be available to anyone who can sign in.

Fit Criterion: For each read channel in `DPR_VisibilityMatrix`, an external supervisor requesting another sponsor's project receives a not-found response and sees no trace of it in lists, searches, counts, or emails. An unverified external supervisor requesting a student profile or a student-proposed project is denied.

`FR_LogExternalErrors` The system shall log all errors caused by external providers and notify university IT professionals and/or course instructors when they are detected.

Rationale: External providers may cause service disruptions and knowing when disruptions happen allows measuring their impact (missed deadlines, for example).

Fit Criterion: An authorized administrator can read a log displaying external provider errors exclusively.

`FR_RejectComponentDisable` The system shall reject all component disable requests if there are pending actions within the component and notify the user of pending actions.

Rationale: Disabling features may potentially leave the system in a partially invalid state. For example, disabling the project approvals component strands pending project approval requests.

Fit Criterion: The user interface displays an error message when attempting to disable the project approval component with at least one record present, and the pending records are displayed.

`FR_DisplayTimestamps` The system shall display a timestamp for any displayed records when they were last updated.

Rationale: Providing users with a timestamp can allow them to judge whether the information they are reading is up-to-date and can contact instructors or supervisors when that is not the case.

Fit Criterion: Any user authorized to read project proposals and user profiles is able

to view a timestamp on all pages.

`FR_DurableNotifications` The system shall store pending notifications durably until delivery is verified. A notification shall be delivered no more than once to each recipient.

Rationale: Lost notifications mean that users are not alerted of potentially time-sensitive matters and this can result in missed deadlines and loss of time.

Fit Criterion: The system being abruptly closed before being able to send a notification sends it after starting back up.

`FR_ReverifyUsers` The system shall re-verify the user before: (1) changing credentials, email, or linked authentication; (2) deleting a proposal, project, profile, thread, or account; (3) changing a user's role or course association; (4) a supervisor or team accepting an agreement; (5) leaving or dissolving a team; (6) a final instructor or TA approval or rejection; (7) confirming an enrollment import or changing course rules or the approval mode. Re-verification must include at least one of: re-entering credentials, entering an email code, responding to an MFA challenge, and/or responding to an SSO challenge.

Rationale: Sessions may get stolen or unsecured devices may get compromised. Actions which are difficult to undo should be protected to ensure the actual account holder is performing the action.

Fit Criterion: The system prompts user to re-enter credentials after requesting to delete a project they proposed.

`FR_ImportEnrollment` The system shall allow instructors and TAs to import a course's enrollment list from Avenue to Learn, show a preview of the students to be added and the differences from the current list, and apply the import only after the user confirms it. The system shall check imported entries for duplicates and malformed identifiers and show any problems in the preview before the import can be confirmed.

Rationale: Populating a course with its students by hand is time consuming, and an unchecked import could wrongly add or exclude students.

Fit Criterion: An instructor or TA can import an enrollment list and see a preview. No change is applied to the course until the user confirms.

`FR_UpdateEnrollment` The system shall allow instructors and TAs to add students to a course from an uploaded file or by manual entry, including students who join after the initial import.

Rationale: Having an Avenue site is optional, Avenue may be unavailable, and students sometimes join a course late.

Fit Criterion: In a course with no Avenue site, the enrollment list can be populated from a file or manually. A student can be added manually after an import.

`FR_EnrollmentDiscrepency` The system shall flag students who are in the course but absent from the latest enrollment list, and shall not remove them from the course or a team until an instructor or TA confirms.

Rationale: Removing students automatically could leave a team undersized or invalid in the middle of approval.

Fit Criterion: A student absent from the latest import is listed for review and remains on their team until course staff confirm the removal.

`FR_NeedsGroup` The system shall show instructors and TAs the enrolled students who are not on a team.

Rationale: Instructors currently compare the class list against team lists by hand to find and contact students without a team.

Fit Criterion: The list contains exactly the enrolled students who are on no team, and it updates as teams change.

`FR_AssignTA` The system shall allow instructors and TAs to assign a TA to each team, and shall show the assigned TA to the team's members and supervisor.

Rationale: Supervisors want to know their teams' TAs, and the assignment is currently kept in a manually edited file.

Fit Criterion: Course staff can assign a TA to a team. The team's members and supervisor see the assigned TA, or "not yet assigned" if there is none.

7 Data Requirements

This section defines what information Keystone must manage to support capstone project proposals, project discovery, team formation, communication, approval workflows, and course administration. It describes major data entities, their attributes, the meaning of each attribute, and the relationships between them. These data requirements are a proposed logical data model based on the functional requirements describing how database technologies will store critical data.

7.1 Major Data Entities

- **User (Parent)**: An individual registered in Keystone including students, supervisors, instructors, and TAs.
- **Student (Child of User)**: Students are users that will be selecting a project and a team for the capstone course.
- **Supervisor (Child of User)**: Supervisors are users that manage one or more capstone teams on their project(s) and they are able to propose projects themselves which

students may work on.

- **Instructor (Child of User)**: Instructors are users who manage one or more capstone course(s) and provide feedback and resources to help students succeed in the course.
- **TAs (Child of User)**: TAs assist instructors in course and team management, including approving team projects, and may be assigned to teams.
- **Administrator permission (`is_admin` on an Instructor or TA)**: Lets the user manage users' course association and roles (`FR_UpdateRoles`), read the external-provider error log (`FR_LogExternalErrors`), and delete data per `DPR_AdministratorDeletionControl`. It is a permission, not a separate role.
- **Course**: A McMaster capstone course supported by Keystone (such as SE, CS, TRON) and includes settings for a capstone course that enables features and implements team formation/ project selection rules. It contains member and enrollment information, including where the enrollment list comes from.
- **Student Profile**: Contains information about a student's background including bio, program, interests, project preferences, skills, experience, work style, and LinkedIn.
- **Team**: A groups of students participating in the same capstone project together.
- **Team Profile**: Contains the collective groups' member bios, collective skills, project interests, goals, and work style.
- **Project**: A supervisor-proposed or student-proposed capstone project that can be described using a title, problem statement/background, objectives, skills/requirements, team deliverables, project impact, supervisor capacity details, level of current completion, hardware or non-hardware, relevant course (SE, TRON, CS), industry, information regarding confidentiality and IP concerns, and additional project terms.
- **Project Application**: Records a student teams interest in supervisor's project including a decision from both parties.
- **Thread**: Discussion containing a post or replies associated with a project proposal, team formation, or project application.
- **Notification**: Contains information communicated by email related to project updates, decisions, discussion activity, urgent deadlines, and outstanding tasks requiring action.
- **Checklist**: Contains a checklist for the user with checklist items for completion.
- **Course Resource**: Can be a course outline, timeline, guidelines, FAQ, past capstone example, or any other course related content that the instructor may post.
- **Audit Log**: A record of each user action, including who took it, what it was, which record it affected, and when (see `NFR_AuditLog`).

7.2 Project and Team Status Lifecycle

Figure 2 shows the statuses a project and the teams working on it pass through, and the events that change them. The left side is the lifecycle of a project proposal and its listing (one per project). The right side is the lifecycle of each team that works on a project (one per team, so a project with capacity greater than one has several). The states in the figure are the only allowed values of the project status and the team-project status in the data dictionary, and every other section uses these names exactly.

- **Project status:** Draft, Proposal submitted, Proposal changes requested, Proposal rejected, Proposal withdrawn, Open, Full, and Closed.
- **Team-project status:** Applied, Supervisor Accepts, Agreed, Team project submitted, Team project under review, Team project changes requested, Team project rejected, Approved, Approved (change under review), and Declined or withdrawn.

A team's approval does not change the project's status. A listed project is Open or Full depending only on capacity. Selection follows *Applied* → *Supervisor Accepts* → *Agreed*. Applying expresses interest and allows discussion; the supervisor's acceptance offers the team a reserved slot. The team then accepts the working terms to establish mutual agreement. It cannot accept before the supervisor.

Capacity. Let C be the supervisor's capacity for the project and R the number of slots reserved. A listed project is *Open* while $R < C$ and *Full* when $R = C$. A team's application does not use a slot. A slot is reserved when the supervisor accepts a team (the state *Supervisor Accepts*), and R increases by one. Subsequent team acceptance changes the state to *Agreed* without reserving another slot. A reserved slot is released, only if one was reserved, when that team is declined or withdraws, when its team project is rejected, or when it dissolves after approval. Releasing a slot, or raising C, makes a Full project Open. A project becomes *Closed* when the selection period ends, whether it is Open or Full. A Closed project with no approved team is *unselected* (see `UC_FollowUpUnselected`).

Review. Proposals and team projects are each reviewed by the course instructor, who can approve, request changes (the proposer or team then resubmits), or reject. In `EXTERNAL` approval mode the instructor or TA records the outcome in Keystone by hand. In `INTEGRATED` mode an approval adapter reads the outcome from the external system. In both cases the platform does not run the review (see `FR_ExternalApproval`).

Changes after approval. A team may submit a change to an approved project. The team's status stays *Approved* (shown as *Approved, change under review*), and the original approval remains in effect, until the instructor accepts or rejects the change. Either outcome returns the team to *Approved*; accepting the change also updates the record.

7.2.1 Entity Relationships

Entity 1	Entity 2	Multiplicity	Description
Student	Course	N : 1	A student is enrolled in exactly one course
Instructor	Course	N : M	An instructor manages one or more courses
TA	Course	N : M	A TA assists in one or more courses
TA	Team	1 : N	A TA may be assigned to many teams. A team has at most one assigned TA
Student	Student Profile	1 : 0..1	A student may have one student profile associated with their account
Student	Team	N : 0..1	A student belongs to at most one team
Supervisor	Project	1 : N	A supervisor may supervise multiple projects
User	Notification	1 : N	A user may receive multiple notifications
User	Checklist	1 : N	A user may have multiple checklists to handle
User	Threads	N : M	A user may have be part of many threads and a thread can contain multiple users
User	AuditLog	1 : N	A user may be responsible for multiple recorded actions
Team	Team Profile	1 : 0..1	A may have at most one shared team profile
Team	Project	N : 0..1	A student team may have at most one project while a project could have multiple teams.
Team	Project Application	1 : N	A student team may submit multiple project applications.
Team	Thread	1 : N	A team may have multiple discussion threads associated with its formation, and each thread belongs to one team.
Project	Project Application	1 : N	A project may have many project applications but each application belongs to a single project.
Project	Thread	1 : N	A project may have many threads but each of those threads belongs to a single project.
Project Application	Thread	1 : 0..1	An application has at most one discussion thread, created when the first post is made. Each such thread belongs to a single application.
Course	Project	1 : N	Many projects belong to one course
Course	Course Resource	1 : N	A course may have many course resources

Table 2: Entity Relationships

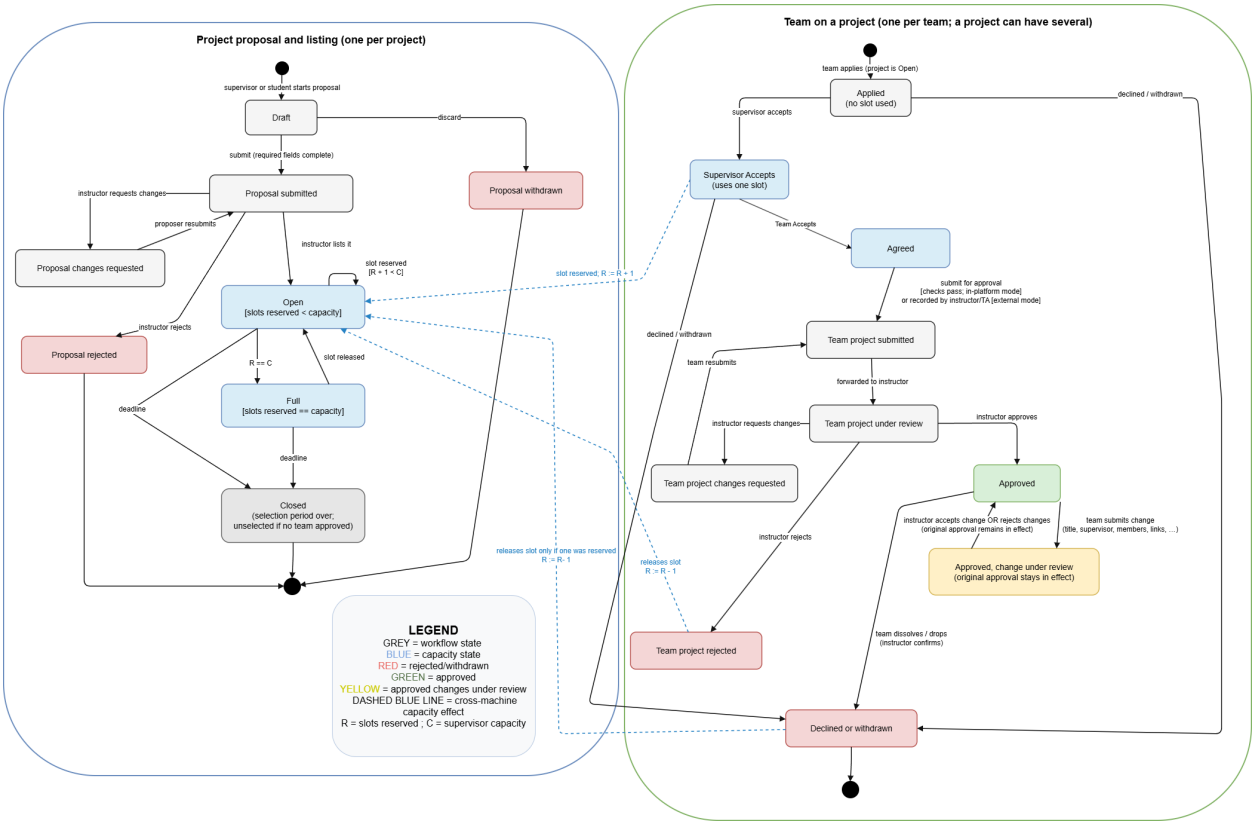


Figure 2: Project and team status lifecycle

7.3 Data Dictionary

The following data dictionary defines attributes of each data entity in Keystone. The data types are conceptual and do not prescribe a particular database implementation. Optional attributes may remain empty depending on the user's role, course configuration, or project workflow. This Data Dictionary is subject to change during implementation.

Table 3: Data Dictionary

Attribute	**Type**	**Description**
User (Parent)		
user_id	ID	Unique ID for each user
name	String	Full name of the user
email	String	Email of the user
user_name	String	MAC ID if signin through SSO or other Auth ID
account_status	Enum	Statuses such as ACTIVE, DISABLED, PENDING

Attribute	Type	Description
is_admin	Boolean	True if the user holds the administrator permission (see Table 1). Only true for an instructor or TA. Only an administrator can change it.
Student (Child of User)		
mac_id	String	McMaster MacID. Equals `user_name` for users who sign in through SSO; `user_name` is authoritative.
student_number	String	McMaster student number, unique per student
enrolled_course	Course	Capstone course in which the student is enrolled.
enrollment_status	Enum	ENROLLED, or FLAGGED if absent from the latest import
Instructor (Child of User)		
mac_id	String	McMaster MacID. Equals `user_name` for users who sign in through SSO; `user_name` is authoritative.
managed_courses	Collection<Course>	Courses that the instructor manages.
TA (Child of User)		
mac_id	String	McMaster MacID. Equals `user_name` for users who sign in through SSO; `user_name` is authoritative.
assigned_courses	Collection<Course>	Courses that the TA assists in.
assigned_teams	Collection<Team>	Capstone teams assigned to the TA.
Supervisor (Child of User)		
supervisor_type	Enum	INTERNAL (McMaster faculty) or EXTERNAL (industry sponsor).
verification_status	Enum	UNVERIFIED or VERIFIED. INTERNAL supervisors are always VERIFIED; EXTERNAL supervisors start as UNVERIFIED.
verified_by_proposal	Project	The approved proposal that verified an external supervisor (empty until verified).
Course		
course_id	ID	Unique ID for each course
course_code	String	McMaster Capstone Course Code

Attribute	Type	Description
course_name	String	Name of the Capstone Course
program	Enum	SE, TRON, CS
instructors	Collection<Instructor>	List of Instructors in charge of the course.
tas	Collection<TA>	List of TAs assisting in the course.
enrollment_list	Collection<Student>	List of students enrolled in the course.
disabled_features	Collection<Enum>	List of features labelled by an enum
min_team_size	Integer	Minimum number of students allowed in a team team.
max_team_size	Integer	Maximum number of students allowed in a team team.
allowed_combinations	Collection<Set<Enum>>	Permitted combinations of SE, CS, and TRON students.
capstone_rules	Collection<String>	Additional descriptive course rules.
approval_mode	Enum	How the course carries out team project approval: IN_PLATFORM, EXTERNAL (instructor or TA records the outcome manually), or INTEGRATED (an approval adapter reads the outcome). See Table 1.
threads	Collection<Thread>	Allow instructors to communicate important information to all supervisors and students in the course.
resource	Collection<CourseResource>	Instructors can post course outline, timeline, guidelines, FAQ, past capstone examples, or any other course related content.
enrollment_source	Enum	AVENUE or FILE; how the enrollment list is populated
last_enrollment_import	DateTime	When the enrollment list was last imported (empty if never)
StudentProfile		
student_profile_id	ID	Unique ID for each student's profile
biography	String	Student Introduction and Background
program	Enum	SE, TRON, CS
skills	Collection<String>	List of skills student has to offer
interests	Collection<String>	List of subjects and areas of interest

Attribute	Type	Description
experience	String	Section on academic, project-based, or professional experience
project_preferences	Collection<Project>	Preferred projects (references) from currently available list.
work_style	String	Description of how the student prefers to work.
linkedin_url	String	Link to student's LinkedIn page.
seeking_team	Boolean	Displays whether the user is seeking a team.
updated_at	DateTime	Date and time when the record was most recently modified.
Team		
team_id	ID	Unique identifier for each team.
team_name	String	Name of the Team.
course	Course	The course that the team is registered in.
members	Collection<Student>	Reference to all team members in the team.
member_roles	Collection<Enum>	All roles of team members.
poc	Student	Reference to the point of contact on the team.
project	Project	The project of the team's application that has reached Agreed or a later status. Empty until then.
status	Enum	Derived, not stored independently: the status of the team's ProjectApplication for the project in `Team.project`, or empty if the team has no project (see Section 7.2). `ProjectApplication.status` is the source of truth.
updated_at	DateTime	Date and time when the record was most recently modified.
ta	TA	Reference to the TA assigned to the team, if any.
threads	Collection¡Thread¿	Discussion threads associated with the team's formation and recruitment.
TeamProfile		
team_profile_id	ID	Unique identifier for each team profile.

Attribute	Type	Description
team	Team	Reference to the team that the profile belongs to.
biographies	Collection<String>	Biographies of each member on the team.
skills	Collection<String>	Combined technical skills of the team.
interests	Collection<String>	Combined interests of the team.
goals	String	General goals working through the capstone project.
work_style	String	The way of working that the team will adopt.
updated_at	DateTime	Date and time when the record was most recently modified.
Project		
project_id	ID	Unique identifier for each project.
title	string	Title of the project (Required).
supervisor	Supervisor	A reference to the supervisor (Optional)
proposer	User	A reference to the person who proposed the project
course	Course	A reference to the course that the project belongs to (Required)
background	String	Main problem statement and background information for the project (Required)
objectives	String	Objectives of the project. (Required)
requirements	Collection<String>	List of skills and experience needed for the project (Optional)
deliverables	String	What deliverables need to be met for the completion of the project (Optional)
impact	String	Expected benefit of the project for its stakeholders (Required)
proposed_solution	String	Potential solution that the team can choose to implement (Optional)
completion_status	String	What work has already been done on the project (Required)
industry	String	Industry or area on which the work is done (Required)
project_type	Enum	HARDWARE or NON_HARDWARE (Required)

Attribute	Type	Description
supervisor_capacity	Integer	Maximum number of teams that can work on the project (Required)
additional	String	Additional Notes or terms for the project (Optional)
confidentiality	String	Confidentiality details. (Required)
ip	String	IP agreements (Required)
status	Enum	Project status: DRAFT, PROPOSAL_SUBMITTED, PROPOSAL_CHANGES_REQUESTED, PROPOSAL_REJECTED, PROPOSAL_WITHDRAWN, OPEN, FULL, or CLOSED (see Section 7.2).
threads	Collection<Thread>	Communication medium on the project.
updated_at	DateTime	Date and time when the record was most recently modified.
ProjectApplication		
application_id	ID	Unique identifier for each project application.
team	Team	Reference to the team that the project application belongs to.
project	Project	Reference to the Project that the application is for.
submission_date	DateTime	When the application was submitted.
status	Enum	Team-project status: APPLIED, SUPERVISOR_ACCEPTS, AGREED, TEAM_PROJECT_SUBMITTED, TEAM_PROJECT_UNDER_REVIEW, TEAM_PROJECT_CHANGES_REQUESTED, TEAM_PROJECT_REJECTED, APPROVED, APPROVED_CHANGE_UNDER_REVIEW, or DECLINED_OR_WITHDRAWN (see Section 7.2).
supervisor_accepted	Boolean	Whether the supervisor has accepted the team. True means a slot is reserved for the team, until the team is declined, withdraws, is rejected, or dissolves.

Attribute	Type	Description
team_accepted	Boolean	Whether the team has accepted the working arrangement after supervisor acceptance. False in APPLIED and SUPERVISOR_ACCEPTS; true in AGREED. May become true only when supervisor_accepted is already true.
thread	Thread	Discussion thread for the application (empty until the first post).
updated_at	DateTime	Date and time when the record was most recently modified.
Thread		
thread_id	ID	Unique identifier for each thread.
author	User	Reference to the user who started the thread with a post.
subject	String	Subject or title of the thread.
content	String	The main content of the thread written by the author.
post_date	DateTime	Creation date of the thread.
replies	Collection<(String (Author), DateTime, String (Content))>	Full discussion and details within the thread.
updated_at	DateTime	Date and time when the record was most recently modified.
Notification		
notification_id	ID	Unique identifier for each notification.
recipient	User	Reference to the user receiving the notification.
type	Enum	Type of Notification like Project, Application, Thread, Due date, etc.
content	String	Information going to the recipient.
date	DateTime	Date that the notification was sent.
delivery_status	Enum	Current status: PENDING, SENT, or FAILED.
updated_at	DateTime	Date and time when the record was most recently modified.
Checklist		
checklist_id	ID	Unique identifier for each checklist.

Attribute	Type	Description
user	User	Reference to the user that has the checklist.
content	Collection<(String (name), DateTime (date), DateTime (completion_date), Enum (status: TODO, INPROGRESS, DONE))>	Items in the checklist.
CourseResource		
resource_id	ID	Unique identifier for each course resource.
course	Course	Reference to the course that has the resource belongs to.
publisher	Instructor/TA	Reference to the Instructor or TA who is publishing the course resource.
title	String	Title for the resource.
type	Enum	Type of resource that the user is posting.
content	Object	Content including information, documents, attachments, calendars, etc.
date	DateTime	Date the resource was published.
updated_at	DateTime	Date and time when the record was most recently modified.
AuditLog		
log_id	ID	Unique identifier for the audit entry.
actor	User (Optional)	User responsible for the action, if applicable.
action	Enum/String	Type of action performed.
entity_type	String	Type of entity affected.
entity_id	ID	Identifier of the affected record.
timestamp	DateTime	Date and time of the event.
change_details	String	Relevant changes or error details, excluding sensitive information.

7.4 Privacy and Data Retention

The following are requirements how the system will manage data to ensure security and privacy in its lifecycle.

7.4.1 Storage

`DPR_CentralizedDataStorage` The system shall securely store its data in a centrally managed backend system accessible by authorized actors

`DPR_DataEntityStructure` The system shall store data based on the Data Entity definitions in Sections 7.1 and 7.3

`DPR_DataAccuracyAndConsistency` The system shall ensure that once a change has been saved, every later request for that data returns the updated version.

`DPR_DailyDataBackups` The system shall back up all data at least once every 24 hours to reduce the risk of data loss from system failures or deletions

7.4.2 Data Retention and Deletion

`DPR_CourseDataRetention` The system shall retain instructor and course related data to be used the following year until the instructor has taken action to make the deletions.

`DPR_AdministratorDeletionControl` The system shall allow deletion of data beyond what `DPR_UserManagedContentDeletion` permits (content the user did not create and is not managing) only by administrators (users with `is_admin` set).

`DPR_UserManagedContentDeletion` The system shall allow authorized users to delete content that they created or are specifically managing on the application including their own personal information as well as make edits.

`DPR_EndOfTermStudentDataDeletion` The system shall automatically delete student information, including copies held in backups, at the end of the capstone period

`DPR_ConsentBasedProjectRetention` The system shall only retain completed project information past the capstone period if the team has consented

7.4.3 Privacy Controls

`DPR_RoleBasedDataPermissions` The system shall implement Role-Based Access Control to allow users to perform read, write, delete, and modify actions based on their role (e.g. Students may not modify other projects that do not belong to them while instructors can edit any project in their course)

`DPR_VisibilityMatrix` The system shall restrict who can read a record according to its type and state, as shown in Table 4, on every channel through which the record can be read.

Record	Who can read it
Proposal in Draft, Proposal submitted, or Proposal changes requested	The proposer, the named supervisor (once they hold a supervisor role other than unverified external supervisor), and the course instructors

Open, Full, or Closed supervisor-proposed project	Students, internal supervisors, TAs, and course instructors. An external supervisor can read only the projects they proposed or are named on (see Table 5).
Student-proposed project published to the student-proposed list	Students, TAs, instructors, internal supervisors, and verified external supervisors
Project application, and threads on a project, application, or team formation	The participants in it, and the course instructors and TAs
Student and team profiles	Students, TAs, instructors, internal supervisors, and verified external supervisors. Not unverified external supervisors.
Team-project status	The team, its supervisor, and the course instructors and TAs
Team project approval feedback	The team, the course instructors and TAs
Course reference material and proposal template	Every signed-in user with a role, including unverified external supervisors
Course administrative records (e.g. enrolled students, list of unselected projects)	Only course instructors and TAs
Name of the TA assigned to a team	the team, its supervisor, and course instructors and TAs

Table 4: Visibility by record type and state

Record	**Unverified external supervisor**	**Verified external supervisor**
Course reference material (outline, calendar, FAQ, guidelines, past project examples)	Yes	Yes
Proposal template	Yes	Yes
Their own proposals and projects, and threads on them	Yes	Yes

Applications to their own projects, and the profiles of the applying teams	Not applicable (no project is listed yet)	Yes
Student and team profiles	No	Yes
Student-proposed projects	No	Yes
Other supervisors' proposals and projects	No	No

Table 5: What an external supervisor can read

7.4.4 Consent

`DPR_TransparencyForRecommendationData` The system shall inform users about all information including personal data that is being retained and used for recommendations

`DPR_PreventUnauthorizedDisclosure` The system shall not disclose any data to unauthorized external parties

`DPR_PrivacyNotice` The system shall provide a Privacy Notice providing users with information related to data collection, use, storage, and retention

`DPR_ConsentWithdrawal` The system shall allow consent withdrawal

7.4.5 Encryption

`DPR_TLSCommunication` The system shall utilize standard and secure network protocols HTTP with TLS to establish secure communication within Keystone

`DPR_McMasterSSO` The system shall utilize McMaster SSO for McMaster student and staff authentication

`DPR_EncryptionAtRest` The system shall encrypt personal information and any confidential data at rest

`DPR_ExternalSupervisorAuthentication` The system shall use another trusted identity provider to authenticate external supervisors

`DPR_NoPlaintextPasswords` The system shall not retain or store any user passwords in plaintext but use securely managed access and refresh tokens to maintain sessions

7.4.6 Compliance

`DPR_FIPPACompliance` The system shall comply with applicable privacy legislation related to collection, use, disclosure, and protection of personal information found under Ontario's Freedom of Information and Protection of Privacy Act (FIPPA).

DPR_McMasterPrivacyPolicies The system shall also adhere to data privacy policies applicable to McMaster and the related faculties

DPR_ThirdPartyAPIAuthentication The system shall ensure secure authentication and communication with APIs and third party services

DPR_SecurityIncidentReporting Unauthorized access, data breaches, data loss or disclosure shall be documented and sent to the relevant parties and the University

7.4.7 Enrollment Data

DPR_MinimalEnrollmentData The system shall import only the enrollment fields it needs (name, email or MacID, program, and course) from Avenue to Learn or an uploaded file.

DPR_EnrollmentDataClassification The system shall treat imported enrollment data as student information, subject to DPR_EndOfTermStudentDataDeletion and DPR_FIPPACompliance.

8 Non-Functional Requirements

8.1 Look and Feel Requirements

8.1.1 Appearance Requirements

NFR_ScreenSize User interface layouts must display correctly on desktop, tablet, and mobile screens.
Rationale: Users will access the platform through phones, computers, tablets, and layouts which do not display correctly may prevent users from performing primary actions.
Fit Criterion: An automated tool such as Playwright renders every page at widths 1920px, 1024px, 430px and reports no horizontal overflows, clipped controls, or controls smaller than a 40px square.

8.1.2 Style Requirements

Not applicable.

8.2 Usability and Humanity Requirements

8.2.1 Ease of Use Requirements

NFR_UsabilityFirstSuccess 75% of first-time users shall be able to access the platform and complete their primary tasks outlined in Section 5 within 5 minutes.

Rationale: No assumptions about users can be made beyond basic web browsing abilities, as described in the user characteristics section (Section 2.4). Many existing web applications that perform similar functions are able to provide an intuitive experience requiring no prior knowledge.
Fit Criterion: A first-time user shall be able to complete the procedure as described in Section 4.2.1 of the Verification and Validation Plan up to the described standard.

8.2.2 Personalization and Internationalization Requirements

Not applicable.

8.2.3 Learning Requirements

`NFR_LearnUnassisted` A first time user shall be able to learn how to perform their primary tasks outlined in Section 5 with no training or assistance.
Rationale: Requiring an upfront, unnecessary time investment to learn how to use the platform can add friction and reduce adoption.
Fit Criterion: User testing outlined in `NFR_UsabilityFirstSuccess` is satisfied with no assistance, prior training, or external learning material.

8.2.4 Understandability and Politeness Requirements

Not applicable.

8.2.5 Accessibility Requirements

`NFR_AccessibilityWCAG` The system shall conform to the WCAG 2.0 Level AA standard.
Rationale: The system will be classified as a public-facing site which operates within Ontario's jurisdiction and must, at least, conform to the minimum standard set by AODA.
Fit Criterion: An automated audit using an industry standard tool (such as Axe or Lighthouse) running on all user interfaces, reporting no critical violations.

8.3 Performance Requirements

8.3.1 Speed and Latency Requirements

`NFR_PerfTail` The system shall respond to user requests with a 95th percentile latency of at most 5.0 seconds.
Rationale: Use is concentrated at the start of the academic year, when projects are proposed and teams are formed (see System Context). The system is expected to have up to 400 staff, students, and external supervisors active at the same time during this period. A slow experience can hurt adoption, and a failure to handle load can lead to

service degradation.
Fit Criterion: A load test simulates 400 concurrent users for a sustained period. Each simulated user repeatedly performs a representative mix of tasks (e.g., browsing the project catalog, viewing a project, applying to a project, replying to a thread) with a 10 seconds pause between actions. The latency of every request is recorded, and the 95th percentile is at most 5.0 seconds. The resulting request rate is measured and reported with the test results, rather than fixed in advance.

8.3.2 Safety-Critical Requirements

Not applicable.

8.3.3 Precision or Accuracy Requirements

Not applicable.

8.3.4 Robustness or Fault-Tolerance Requirements

`NFR_Uptime` The system shall be available with at least 99% uptime ($\sim$ 1.7 hours of unscheduled downtime per week).
Rationale: The system being inaccessible can result in missed deadlines, data loss, and overall frustration. This can impact adoption, perception, and erode trust.
Fit Criterion: The system responds to a periodic health check, which runs every minute, with success responses accounting for at least 99% over any 7-day period. Scheduled maintenance windows are excluded from the measurement.

`NFR_Recoverable` The system shall be recoverable after an incident to a state no more than 24 hours older than the incident. Only if every recovery point within that window is unavailable or fails to restore, the system shall recover from the most recent available recovery point outside the window.
Rationale: The system might experience a data loss incident. It must be recoverable with minimal losses, and a failed or missing recent backup should not leave the system unrecoverable.
Fit Criterion: Using the interface, the system can be restored to a state no more than 24 hours older than a simulated incident. When every recovery point within the window is made unavailable, the system restores the most recent recovery point outside the window and tells an administrator that the window was not met.

8.3.5 Capacity Requirements

`NFR_PeakUsers` The system shall be capable of handling at least 400 concurrent users while meeting `NFR_PerfTail`.
Rationale: Users will be accessing similar resources concurrently, especially at the start of a year. System degradation can prevent users from performing essential tasks.

Fit Criterion: The system shall meet the thresholds specified in NFR_PerfTail with 400 simulated users.

8.3.6 Scalability or Extensibility Requirements

Not applicable.

8.3.7 Longevity Requirements

Not applicable.

8.4 Operational and Environmental Requirements

8.4.1 Expected Physical Environment

Not applicable.

8.4.2 Wider Environment Requirements

Not applicable.

8.4.3 Requirements for Interfacing with Adjacent Systems

NFR_Browsers The system shall be accessible on the latest two major versions of Firefox, Safari, and Chrome.
Rationale: Web browsers provide an abstraction layer that isolates functionality from the underlying operating system environment, removing OS-support as a concern.
Fit Criterion: All target browsers shall be tested using an automated web testing tool such as Playwright or Selenium and complete all tests successfully. This is described in-depth in Section 4.2.3 of the Verification and Validation Plan.

NFR_Container The hosted component of the system shall run on the latest two supported major versions of Docker.
Rationale: Similar to web browsers, Docker provides an abstraction layer over the underlying operating system and a sandboxed environment for executing code.
Fit Criterion: The hosted component of the system successfully builds and starts as a Docker image.

8.4.4 Productization Requirements

NFR_DeployUnassisted The system shall be deployable by an IT professional by following the deployment guide with no assistance in under 30 minutes.
Rationale: Issues during deployment can increase downtime which results in frustration with the platform.
Fit Criterion: A technically competent individual outside of the main development

team is able to complete the task described in Section 4.2.2 of the Verification and Validation Plan within the time constraint.

8.4.5 Release Requirements

Not applicable.

8.5 Maintainability and Support Requirements

8.5.1 Maintenance Requirements

`NFR_ChangeEffort` The effort required to make likely changes, which can be found in Section `LC_LikelyChanges`, shall be less than 20% of the original development time.
Rationale: A system which is difficult to maintain will eventually reach an unusable state when environments change (e.g.: hardware changes or software updates introduce/expose bugs, vulnerabilities are discovered, privacy regulations change, etc.) and fixes/updates are too costly to perform.
Fit Criterion: Based on a development effort of 40 hours / week, maintenance cost shall not exceed 8 hours / week.

8.5.2 Supportability Requirements

Not applicable.

8.5.3 Adaptability Requirements

`NFR_LiveDisable` All major features, with the exception of the RBAC component, can be disabled on a live system with no code changes.
Rationale: The need to toggle major features was established during stakeholder interviews. The platform needs to be flexible to fit the working style of a variety of instructors. RBAC ensures visibility rules are enforced and this is essential for protecting sensitive information.
Fit Criterion: Every major feature can be disabled on a live system with no code changes.

8.6 Security Requirements

8.6.1 Access Requirements

`NFR_Authenticate` The system shall require authentication before any other actions are possible.
Rationale: Authentication provides the system with a user's identity which ensures access rules are enforced.
Fit Criterion: Unauthenticated users receive an error response when making requests which are not authentication-related.

`NFR_Authorize` The system shall restrict access only to students, TAs, and instructors directly enrolled in the capstone courses, and to potential supervisors approved by an instructor. All users are only able to access data that they are approved to view. External supervisors can access only the data permitted to them in Table 4.
Rationale: The system stores sensitive user and course data which is a primary target of cyber attacks.
Fit Criterion: Authenticated users receive an error response when requesting or modifying data they are not authorized to view or modify.

8.6.2 Integrity Requirements

`NFR_ConcurrencySafety` All data operations involving multiple data entities must be concurrency-safe.
Rationale: Many data operations will be executing at any given time and the system must ensure all data operations do not operate on stale or corrupt data.
Fit Criterion: The system remains in a correct state after a set of concurrent data modification operations. A correct state is one which can be represented as the output of some sequence of a set of concurrent operations.

8.6.3 Privacy Requirements

`NFR_EssentialPersonal` The system shall only collect personal data when it is essential to the system's operations.
Rationale: The platform stores personal data to facilitate basic operations and is subject to regulations imposed by Ontario's FIPPA.
Fit Criterion: All stored user data can be traced to a requirement in Section 6 or Section 8.

`NFR_EncryptTransit` The system shall ensure data is encrypted in-transit.
Rationale: Communications can be intercepted by other peers in a network and internet service providers (ISPs) and hold sensitive data. Encryption ensures that data can only be read by the server and client.
Fit Criterion: The system refuses to respond to all requests made over an unencrypted connection.

`NFR_EncryptRest` The system shall ensure data is encrypted at rest, with decryption protected by a cloud key management provider or a hardware security mechanism.
Rationale: While physical access to server storage is unlikely, a system vulnerability in the host operating system, virtualization layer, filesystem, or in other dependent systems may allow attackers to read stored user data.
Fit Criterion: Database content is not readable and not recoverable from a database fragment/part file without a decryption key.

`NFR_PasswordHash` All user passwords shall be stored in a cryptographically secure, salted, and irreversible format.

Rationale: Ensuring passwords are not recoverable protects users from data exposure events.
Fit Criterion: Passwords are hashed using Argon2id with a random salt generated for each password.

8.6.4 Audit Requirements

`NFR_AuditLog` All user actions shall be audited including metadata such as when the actions where taken, what action was taken, the user, and their privilege level at the time of the action.
Rationale: User actions must be auditable to ensure any discrepancies about events such as team enrolment, project approvals, and other critical events are tracable.
Fit Criterion: A full sequence of audit logs can be composed together to provide a 100% accurate image of the system's state.

`NFR_AuditTamper` All logs shall be stored in tamper-resistant/tamper-evident storage.
Rationale: Audit logs may be the target of attackers aiming to remove any trace of their activities.
Fit Criterion: Attempting to tamper with or remove audit log storage files raises an alert to system operators.

8.6.5 Immunity Requirements

Not applicable.

8.7 Cultural Requirements

8.7.1 Cultural Requirements

Not applicable.

8.8 Compliance Requirements

8.8.1 Legal Requirements

`NFR_ComplianceAODA` The system shall comply with AODA regulations, as specified in `NFR_AccessibilityWCAG`. **Rationale**: The system will be hosted in Ontario and classified as a public-facing site. It must comply with the applicable accessibility regulations. **Fit Criterion**: It meets the criterion set in `NFR_AccessibilityWCAG`.

8.8.2 Standards Compliance Requirements

Not applicable.

9 Requirements Traceability Matrices

Functional Req	Stakeholder Req	Use Case
FR_SupervisorProposal	SR_ProposalTemplate	UC_SubmitProposal
FR_ProposalOptionalFields	SR_OptionalFields	UC_SubmitProposal
FR_StudentProposal	SR_StudentProposals	UC_ProposeOwnProject
FR_ReviewProposal	SR_HumanApproval	UC_ReviewProposal
FR_ViewApprovedProjects	SR_BrowseProjects, SR_SinglePlace	UC_BrowseProjects
FR_ViewProjectStatus	SR_BrowseProjects, SR_ProjectStatus	UC_BrowseProjects
FR_StudentProfile	SR_StudentProfiles	–
FR_StudentTeams	SR_SinglePlace	–
FR_ValidateTeamRules	SR_TeamValidity	UC_ValidateTeams
FR_TeamDiscovery	SR_SinglePlace	–
FR_TeamProfile	SR_StudentProfiles, SR_TeamContacts	UC_TeamProfile, UC_ViewTeam
FR_TeamRecommend	–	–
FR_ProjectInterest	SR_SinglePlace	UC_SelectProject
FR_DeclineTeam	SR_SinglePlace	UC_SelectProject
FR_ProjectThreads	SR_ReviewDialogue, SR_SinglePlace	UC_ReviewProposal
FR_ThreadsNotify	SR_StatusNotify	UC_NotifyChecklist
FR_RequestApproval	SR_ReviewDialogue, SR_HumanApproval	UC_ApproveTeamProject
FR_ApprovalResponse	SR_HumanApproval, SR_ReviewDialogue, SR_TAApproval	UC_ApproveTeamProject
FR_ExternalApproval	SR_HumanApproval	–
FR_ApprovalPreflightValidate	SR_TeamSubmissionChecks	UC_ApproveTeamProject
FR_ApprovalRevision	SR_ReviewDialogue	–
FR_DisableFeatures	SR_CourseConfig	–
FR_ReferenceMaterial	SR_ProposalGuidance, SR_CourseInfoEarly, SR_CrossCourseTimelines	UC_SubmitProposal, UC_ViewTimelines
FR_RequiredActionsList	SR_Checklist	UC_NotifyChecklist
FR_ActionRequiredEmail	SR_PushNotify, SR_Checklist	UC_NotifyChecklist
FR_ReadCourseInformation	SR_UnselectedProjects, SR_TeamValidity	UC_ValidateTeams, UC_FollowUpUnselected
FR_ExportCourseInformation	SR_ReduceAdmin, SR_TeamValidity	UC_ValidateTeams

FR_MacSSO	SR_ExternalSupervisorAccess	–
FR_AuthorizationRBAC	SR_ExternalSupervisorAccess	–
FR_UpdateRoles	SR_ExternalSupervisorAccess	–
FR_ExternalSupervisorVerification	SR_ExternalSupervisorAccess	UC_SubmitProposal
FR_ExternalSupervisorScope	SR_ExternalSupervisorAccess	UC_SubmitProposal, UC_ViewTeam
FR_LogExternalErrors	–	–
FR_RejectComponentDisable	–	–
FR_DisplayTimestamps	–	–
FR_DurableNotifications	SR_PushNotify, SR_StatusNotify	UC_NotifyChecklist
FR_ReverifyUsers	–	–
FR_ImportEnrollment	SR_Enrollment	UC_ImportEnrollment
FR_UpdateEnrollment	SR_Enrollment	UC_ImportEnrollment
FR_EnrollmentDiscrepency	SR_Enrollment, SR_TeamValidity	UC_ImportEnrollment
FR_NeedsGroup	SR_Enrollment, SR_TeamValidity	UC_ImportEnrollment, UC_ValidateTeams
FR_AssignTA	SR_TeamContacts	UC_ViewTeam

Table 6: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req	Nonfunctional Req
FR_SupervisorProposal	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted, NFR_ConcurrencySafety
FR_ProposalOptionalFields	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted, NFR_ConcurrencySafety
FR_StudentProposal	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted, NFR_ConcurrencySafety
FR_ReviewProposal	NFR_Authorize, NFR_AuditLog
FR_ViewApprovedProjects	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted, NFR_PerfTail, NFR_PeakUsers
FR_ViewProjectStatus	NFR_ConcurrencySafety
FR_StudentProfile	NFR_Authorize, NFR_EssentialPersonal, NFR_EncryptRest

FR_StudentTeams	NFR_ConcurrencySafety, NFR_AuditLog
FR_ValidateTeamRules	NFR_ConcurrencySafety
FR_TeamDiscovery	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted
FR_TeamProfile	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted, NFR_Authorize
FR_TeamRecommend	NFR_ChangeEffort
FR_ProjectInterest	NFR_ConcurrencySafety
FR_DeclineTeam	NFR_ConcurrencySafety, NFR_AuditLog
FR_ProjectThreads	NFR_Authorize, NFR_AuditLog
FR_ThreadsNotify	NFR_Uptime
FR_RequestApproval	NFR_ConcurrencySafety, NFR_AuditLog
FR_ApprovalResponse	NFR_Authorize, NFR_AuditLog
FR_ExternalApproval	NFR_LiveDisable, NFR_ConcurrencySafety, NFR_AuditLog
FR_ApprovalPreflightValidate	NFR_ConcurrencySafety
FR_ApprovalRevision	NFR_ConcurrencySafety, NFR_AuditLog
FR_DisableFeatures	NFR_LiveDisable, NFR_Authorize
FR_ReferenceMaterial	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted, NFR_Authorize
FR_RequiredActionsList	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted, NFR_ConcurrencySafety
FR_ActionRequiredEmail	NFR_ConcurrencySafety, NFR_Uptime
FR_ReadCourseInformation	NFR_Authorize, NFR_EssentialPersonal
FR_ExportCourseInformation	NFR_Authorize, NFR_ConcurrencySafety
FR_MacSSO	NFR_Authenticate, NFR_EncryptTransit, NFR_PasswordHash
FR_AuthorizationRBAC	NFR_Authorize, NFR_EssentialPersonal
FR_UpdateRoles	NFR_Authorize, NFR_AuditLog
FR_LogExternalErrors	NFR_Uptime, NFR_AuditLog, NFR_AuditTamper
FR_RejectComponentDisable	NFR_LiveDisable, NFR_ConcurrencySafety
FR_DisplayTimestamps	NFR_AuditLog
FR_DurableNotifications	NFR_Recoverable, NFR_ConcurrencySafety
FR_ReverifyUsers	NFR_Authenticate, NFR_PasswordHash
FR_ImportEnrollment	NFR_ConcurrencySafety, NFR_EssentialPersonal, NFR_AuditLog
FR_UpdateEnrollment	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted, NFR_ConcurrencySafety

FR_EnrollmentDiscrepency	NFR_ConcurrencySafety, NFR_AuditLog
FR_NeedsGroup	NFR_Authorize
FR_AssignTA	NFR_Authorize, NFR_ConcurrencySafety
FR_ExternalSupervisorVerification	NFR_Authorize, NFR_AuditLog
FR_ExternalSupervisorScope	NFR_Authorize, NFR_EssentialPersonal

Table 7: Traceability Between Functional and Nonfunctional Requirements

10 Rationale

10.1 Scope

The scope was decided based on an estimate of what stakeholders need and what would be considered nice-to-have features, as described in Section 1.2.

Keystone focuses on project proposals, discovery, team formation, visibility, and project approvals rather than the entire capstone lifecycle. Features such as automated project evaluations, AI-features, full course management, post-approval supervision, meeting scheduling, messaging, and intellectual property dispute resolution were excluded to keep the project scope manageable and most were not desirable implementations for the stakeholders or were simply overkill for this project. Also, too much automation was undesirable by stakeholders as instructors prefer to be in control of approving projects and manually providing feedback.

10.2 Functional Requirements

Each functional requirement's rationale can be found underneath the corresponding requirement in Section 6. The functional requirements were generally derived from stakeholder requirements, as demonstrated in Section 9. Therefore, the core functionality of the system is determined by the needs of its users. Functional requirements without direct stakeholder traceability primarily address underlying system and security needs necessary to provide a reliable platform.

10.3 Non-Functional Requirements

Each non-functional requirement's rationale can be found underneath the corresponding requirement in Section 8. These requirements establish the quality standards necessary for Keystone to serve its stakeholders effectively. Requirements such as access, availability, accessibility, and usability ensure that stakeholders can use the system efficiently and without significant challenges. While functional requirements define the system's capabilities, non-functional requirements provide measurable goals for its quality, robustness, and performance.

10.4 Data Requirements

The data requirements in Section 7 were established to support the functional requirements while maintaining consistent and accurate information across the platform.

- **User Child Entities:** A parent User entity was defined with child Student, Supervisor, Instructor, and TA entities to represent shared information while distinguishing role-specific attributes. This was especially necessary when restricting entity relations (e.g. only Students belong to a team)
- **Entity Choices:** They come directly from the functional and stakeholder requirements as they determine all of the objects needed to represent the required actors and features in the system.
- **Course Configuration:** Course-specific settings and rules are associated with the Course entity to support different capstone workflows without requiring structural changes to the data model.
- **Data Integrity:** Relationships and constraints were established to prevent invalid team memberships, duplicate project assignments, and inconsistent records.
- **Data Privacy and Retention:** These requirements were designed to ensure that a user's sensitive information such as email and any data about their project remains secure in our backends and in compliance with Privacy Protection policies. Keeping data encrypted can do this and RBAC keeps everyone's access adhering to the principle of least privilege.

Data Visibility and Hosting: The CS instructor asked whether data can stay internal to McMaster when external supervisors are involved, and whether internal-only hosting would prevent those supervisors from reaching it. The data in Keystone falls into four groups, and none of them needs to be confined to the university network.

- *Reference material* (course outline, expectations, FAQ, past project examples, and the proposal template) is not sensitive, and every signed-in user can read it, including an unverified external supervisor.
- *Student and team profiles* contain only what students choose to enter, which is comparable to a professional profile on Linkedin or their personal websites. They are visible to course members, internal supervisors, and verified external supervisors, and users are informed of this through the privacy notice (`DPR_PrivacyNotice`).
- *Industry project proposals* are confidential to their sponsors. They are protected by access control, not by hosting location: an external supervisor sees only their own proposals, and other supervisors never see them (`FR_ExternalSupervisorScope`).
- *Administrative records* (e.g. the imported enrollment list, approval feedback, and the audit log) are restricted by role and are never visible to external supervisors. A super-

visor sees only the status of their team's project and the name of the team's assigned TA.

Because the data that external supervisors can reach is public in nature, self-published by students, or protected by role-based access, Keystone does not need to be restricted to the university network, and doing so would stop industry sponsors from using it. Instead, access depends on who signs in. A McMaster sign-in is accepted as proof of affiliation. A sign-in through the external provider proves nothing, so it starts *unverified* with access to reference material only, and becomes *verified*, gaining access to student profiles and student-proposed projects, only when an instructor approves a proposal that the user submitted (FR_ExternalSupervisorVerification). Users also accept terms of use and a privacy notice at sign-in, which informs them of what is visible to whom. This supplements access control and FIPPA compliance (DPR_FIPPACompliance) and does not replace them. Where Keystone is hosted does not change who may see which records. The protection of personal information on any host is provided by the encryption and privacy requirements (DPR_EncryptionAtRest, DPR_TLSCommunication, DPR_FIPPACompliance).

10.5 AI Requirements

Implementing an AI-based solution is primarily outside the core project scope and is instead considered a stretch goal. The system does not require training an AI model to fulfill its primary functionality.

11 Likely Changes

LC_Users **Role Permissions.** What each role may do may change. The TA role in particular was defined late and may be refined as more instructors are interviewed, and additional roles may be needed for people outside the main stakeholders, such as maintainers. *Affected Entities:* User, Student, Instructor, Supervisor, TA. *Related Requirements:* FR_MacSSO, FR_AuthorizationRBAC, FR_UpdateRoles.

LC_TeamCourseRestrictions **Course and Team Rules.** The team formation and eligibility rules may change. The CS instructor has not yet been interviewed, so CS team sizes and program restrictions are unconfirmed, and students from other departments (e.g., ECE or iBioMed students on TRON projects) may need rules beyond team size and program combinations. *Affected Entities:* Student, Team, Course. *Related Requirements:* FR_ValidateTeamRules, FR_DisableFeatures.

LC_ProposalTemplates **Proposal Template Fields.** The required and optional fields of the project proposal may change as instructors review the template. Courses may need different fields (for example hardware information for TRON), and instructors want standards that do not discourage supervisors from proposing projects. *Affected Entity:* Project. *Related Requirements:* FR_SupervisorProposal, FR_ProposalOptionalFiel

FR_StudentProposal.

LC_ApprovalWorkflows **Approval Process.** How a course approves team projects may change. The TRON course approves a project when its team is given a group number, and the CS process is unconfirmed, so a course may need different approval steps or approvers than the ones specified here. *Affected Entities:* Project, Project Application, Course. *Related Requirements:* FR_RequestApproval, FR_ApprovalResponse, FR_ExternalApproval, FR_DisableFeatures.

LC_Recommendations **Recommendations.** The recommendation requirement may be removed or made switchable if instructors judge that its benefit does not justify the overhead, or expanded if stakeholders want matching on additional criteria such as skills. Feedback from instructors and usability testing would trigger either change. *Affected Entities:* Student Profile, Team Profile, Project. *Related Requirement:* FR_TeamRecommend.

LC_SignIn **Sign-In and Re-Verification.** How users sign in, and which actions require a user to re-verify their identity, may change as security requirements change. This could be due to incidents, legal changes, or other external factors. *Affected Entity:* User. *Related Requirements:* FR_MacSSO, FR_ReverifyUsers.

LC_CourseFeatures **Course Features and Settings.** The set of features an instructor can turn off, and the settings a course can configure, may change as features are added and as more courses (such as CS) adopt the platform with different needs. *Affected Entity:* Course. *Related Requirements:* FR_DisableFeatures, FR_RejectComponentDisable.

LC_ProfileFields **Profile Fields.** The information collected in student and team profiles may change. Fields may be added if matching uses more criteria (LC_Recommendations), or removed if they are not essential (NFR_EssentialPersonal). *Affected Entities:* Student Profile, Team Profile. *Related Requirements:* FR_StudentProfile, FR_TeamProfile.

LC_NotificationRules **Notification Rules.** Which events cause a notification, who receives it, and how it is delivered may change. Stakeholders want to be notified through channels they already use (SR_PushNotify), and may later ask for notification preferences or other channels. *Affected Entity:* Notification. *Related Requirements:* FR_ThreadsNotify, FR_ActionRequiredEmail, FR_DurableNotifications.

LC_ThreadTargets **Discussion Thread Targets.** Which records discussion threads can be attached to, and how many threads each may have, may change. For example, instructors may need course-wide threads to communicate with all supervisors and students in a course. *Affected Entities:* Thread, Project, Project Application, Team, Course. *Related Requirement:* FR_ProjectThreads.

LC_RequiredActions **Required Actions.** Which actions appear on a user's list of required actions (SR_Checklist) may change as features are added which con-

tain actions deemed as required. *Affected Entity:* Checklist. *Related Requirements:* FR_RequiredActionsList, FR_ActionRequiredEmail.

LC_CourseReports **Course Reports.** The contents of course reports, and the format they are exported in, may change. Each course tracks approval differently (TRON assigns group numbers), so instructors may need different information, or a format their own tools can read. *Affected Entity:* Course. *Related Requirements:* FR_ReadCourseInformation, FR_ExportCourseInformation.

12 Unlikely Changes

- **Core Stakeholder Roles.** The general stakeholder roles will not be changed since they are the primary users of the application (Students, Instructors, Supervisors, TAs).
- **Human-centered approval.** Instructors explicitly stated that there are aspects where they do not desire automation and therefore, the manual approval and feedback system will not change.
- **Course Organization.** Course Organization where students, instructors, projects, and teams will remain with their associated course and their respective roles/relationships in this environemnt will not change.
- **Vision and Overall Flow.** The general workflow will remain unchanged: students may browse projects at any time, with or without a team, and form teams based on their interests. A student must be in a team to apply for or propose a project, and the team's project then obtains instructor approval.

13 Appendices

13.1 Appendix A: Additional Materials

13.1.1 Stakeholder Interview Notes

The notes below were taken during stakeholder interviews. Each note is numbered with its stakeholder tag (S, ES, IS, SEI, TI, CI) so that stakeholder requirements can reference the specific note(s) they came from. Each role was covered by a small number of interviews, summarized in Table 8, so the broad claims in this document should be validated as more stakeholders are consulted.

Role	Tag	Interviews
Student	S	1 interview, with 1 student
External supervisor	ES	1 joint interview, with 2 supervisors
Internal supervisor	IS	1 interview, with 1 professor
SE course instructor	SEI	1 interview
TRON course instructor	TI	1 interview
CS course instructor	CI	1 interview

Table 8: Stakeholder interviews

Student (S) Student who chose a project from the list of given projects.

`S_TeamFirst` Formed a team with classmates they already knew over the summer, and had project ideas before the term started. In hindsight, picking the project before the teammates might have been better, since teammates with shared interests working on their preferred project could lead to a better experience.

S2Each member chose their top 5 projects after reading all of them, then the team settled on a top 3 in case some were unavailable.

`S_Vague` Project descriptions were often vague. Some were ignored and only the title was read.

`S_ProjectInfo` Information on hardware, software stack, and industry/field of study would be helpful.

S5One supervisor held a meeting with all interested groups right before the project selection deadline, which added uncertainty.

`S_Competition` Another group was competing for the same project, and losing it led to frustration.

`S_Uncertainty` Considerable uncertainty while landing a project, with worry about not having one available.

`S_OwnTemplate` A template for proposing their own project would help.

`S_OwnGuidelines` Guidelines or an FAQ on proposing their own project would reduce uncertainty about size, scope, and content requirements.

`S_Status` Wants a way to know when a project is taken, open, dropped, etc.

`S_Newsletter` A notification mechanism for project status would help; a newsletter was suggested.

`S_MeetingUnclear` Supervisor communication (mechanism, process, content) is unclear: is an agenda needed, will the supervisor mostly speak, who leads each meeting? Guidelines may be enough.

S13Unsure whether supervisors will have time to meet regularly (e.g., weekly).

`S_MeetingPurpose` Meetings are useful for answering a backlog of questions and filling knowledge gaps with an expert instead of researching alone.

`S_EmailSimple` Prefers email for simple questions.

`S_SuggestedMeetings` "Suggested meetings" aligned with deliverables would set expectations and help groups know whether they are on track.

S17The GitLab approval process was intuitive due to prior GitHub/GitLab experience.

S18JSON files were a hassle but not problematic.

`S_ManyTabs` Finding where things are is time consuming and requires many open tabs.

S20Google Calendar was helpful but would benefit from more information.

`S_OnePlace` Having everything in one place (like Avenue) would have been more helpful.

External Supervisor (ES) Supervisor from outside the university.

ES1The proposal process was smooth, and working with one of the professors and meeting the team was smooth as well.

`ES_StatusHard` Knowing project statuses is a challenge.

`ES_SharedNotes` A centralized platform for meeting notes would help. Some teams get more information from meetings than others, and notes should be shared with everyone. Summaries of meetings and project status, and a template for them, would be great.

`ES_Schedule` A schedule would clear up expectations. Some teams come in early and some late; timelines and deadlines are unclear.

`ES_Template` A template would be better than an open-ended description box for proposals. It is hard to know what is expected from the capstone team, so a best-effort approach is taken.

`ES_CourseInfo` Guidelines on course content, content coverage, and capstone objectives would let supervisors tailor the experience. The course calendar and outline are wanted earlier, ideally before proposing.

`ES_Examples` Examples (e.g., past capstones) would help with idea generation and expectation setting. Ideas were thrown around and refined internally, but guidance would have been better.

ES8Extra back-and-forth with the professor (capstone processes, expectations, IP questions) could be avoided by providing more information up front; a template or FAQ would help.

`ES_NotifyNotRemember` Being notified to do tasks on a platform is fine; having to remember to open it is the problem. Email or phone notifications should be considered. A notification and checklist is preferred over searching in the platform.

`ES_ManyLogins` Too many platforms to log in to is tedious. A new application is fine.

`ES_SuggestedTimes` Suggested meeting times and a schedule would help, though some supervisors have their own schedule that works better.

`ES_NoStudentInfo` No information about students before team meetings; a leap of faith was required, relying on gut feeling and the right questions. Did not know they had the final say.

`ES_StudentProfiles` Wants a way to learn about students (team bio, interests, skills, experience), even if not all shown up front; looking them up (e.g., LinkedIn) was mentioned.

`ES_Matching` Wants teams matched to projects based on preference criteria and skill sets.

`ES_WhoToEmail` Hard to find where to communicate back to students. Groups usually had one representative email and CC their members, which made emails hard to search. With many names and no group numbers, they defaulted to the first person who emailed. Inviting everyone was too taxing; wants some way to streamline.

`ES_Admin` Overall, there is a lot of administration on the supervisor side.

Internal Supervisor (IS) Professor at the university who supervises teams, submits proposals, and advises the instructors.

`IS_Role` Mostly a supervisor. Submits proposals and then waits to see what else is on the list in case something interests him. Interviews and supervises groups, attends

lectures when possible, helps with presentations and Expo posters, and is often on the outside of the course.

`IS_StudentProposed` Unsure how student-proposed projects will be handled.

`IS_ApprovalTiming` The three capstone courses are not well coordinated. The timing and method of the approval process (whether a project can be a capstone project) is not really known.

`IS_Standardize` Current descriptions are 100–200 words. Standardizing them is hard and he is unsure what the standard should be. Some supervisors may want something specific, and one group delivered something related to, but very different from, the proposal.

`IS_Schedule` Supervisors do not know the course schedule (when the SRS is due, when implementation starts, validation testing, user testing). There is confusion about what happens and when things are due.

`IS_MultiCourse` Supervisors with teams across multiple courses (SE/CS/Tron) may not know that each course has different timelines and requirements.

`IS_Procedure` No standardized procedure, e.g., how a group with an idea finds a supervisor. Wants the timeline and procedure centralized and standardized.

`IS_Reports` Wants a low-effort mechanism for supervisors to submit a report or feedback at the end. One professor sends feedback emails, but others do not seem to remember them and send reports voluntarily.

`IS_Communication` Communication and regular updates (check-ins) between supervisors and teams are essential and need to be accounted for. Discussion is needed on weekly meetings and who questions go to (TA or professor).

`IS_KnowTA` Wants to know who each of his groups' TAs are.

`IS_StudentSkills` Student skills would help guide discussions with supervisors.

`IS_Scope` Would like the capstone process all in one place, but is unsure whether that would work with other professors, or whether the platform should cover only the proposal stage.

`IS_Accounts` Accounts are mentioned as a need (details not captured in notes).

Software Engineering Instructor (SEI) Professor who runs the Software Engineering capstone course.

`I_CurrentProcess` Current process: before the course, emails contacts to find projects. Interested supervisors submit an MS Form (title, organization, contact, description, confidentiality, IP concerns). A Google Form funnels into a spreadsheet that a script

turns into LaTeX. Students form teams, choose projects, and organize meetings. A merge request in GitLab counts as approval.

`I_SupervisorChanges` Supervisor details often change and are updated. Supervisors are contacted to confirm they know a team will work with them, which often surfaces Expo and other capstone details.

SEI3Teams are classified by project hardware (SE vs. Mech).

`I_TeamJSON` Team details are recorded in a JSON file, processed and verified by a Python script (e.g., every person is in exactly one team). The team composition CSV should be an output that is up to date, correct, and valid.

`I_Decisions` Would rather spend time on decisions such as project viability and approval.

`I_Recruiting` Often has to reach out to potential supervisors to have enough projects each year.

`I_Unselected` Wants to know which projects have not been selected, e.g., to send a thank you email.

`I_Automate` Coordination, communication, and details such as approved projects and groups should be automated as much as possible.

`I_ApprovalCriteria` Approval criterion: enough substance. Game projects often need extra work to add substance (e.g., more analysis such as proving fairness, or more components).

`I_Standards` Standards could be considered, but enforcing them can scare away good projects; optional questions may help.

`I_NoAIApproval` No AI agents making approvals. AI-generated summaries could be considered, but their usefulness is unclear.

`I_PRs` Pull requests are still favored because they expose students to CI and show its usefulness, and allow back-and-forth between professor and students.

`I_RepeatFeedback` The same feedback is sent to multiple groups. An AI agent could respond to PRs with already-resolved issues (e.g., professor not added as collaborator), leaving more important work to the instructor.

`I_Universal` Possibly usable universally, which may require modularity and extensibility.

`I_MatchingOptional` Algorithmic matching of teams to projects could be interesting, with a way to turn it off. Overhead must be worth it; this could be offset by moving other things (e.g., meetings) from Teams or Zoom onto this platform.

Tron Instructor (TI) Professor who runs the Mechatronics capstone course.

`TI_Course` The TRON course was separated from the SE course because of growing class size. The TRON and CS courses run very differently, and he runs his course more flexibly than the SE course. He emphasizes personal responsibility and meets with each team informally every week.

`TI_Tools` Most of the course runs on Microsoft Teams, with a private channel per team that the instructor and TAs can access, plus general channels and shared files (lectures, IP information, example videos, rubrics). Avenue holds grades, submissions, and a list of deliverables. Deliverables are due at the end of the week and are accepted without penalty until the following Tuesday.

`TI_Similar` The beginning of the capstone courses is the most common part between them, and the courses have a similar number of deliverables.

`TI_Approval` Projects are approved on Teams. A project is approved when its team receives a group number.

`TI_Roster` He receives the list of students in the capstone courses, manually marks off those who are in a group, and contacts the rest. He tries not to force students into groups, so some students are delayed in getting a group. One student joined a group very late, and the situation was handed to the undergraduate advisor.

`TI_Populate` Teams channels must be created and populated from Avenue or added manually, and students are removed if they are not on Avenue. Populating a platform with student information is not trivial. Having an Avenue course is optional; an instructor requests one and it is then created.

`TI_CrossProgram` SE students on TRON projects are treated as TRON students, and their grades are sent to the SE instructor. TRON students cannot join software-only projects. ECE students on TRON projects must be graded by an ECE professor to show their involvement. Cross-discipline capstones, such as iBioMed, need additional considerations and involvement from professors, and accreditation is a concern.

`TI_Projects` Projects are collected over the summer through the SE instructor and his contacts, and are kept in a LaTeX document. Many TRON projects are student-proposed, and student-proposed projects are often high quality and people-centred.

`TI_StudentOwned` The capstone is student-centred. Students own the project even if stakeholders are involved, and the only rule is that students must be able to talk about it (presentations, EXPO). Supervisor-only deliverables are secondary and should not overwhelm students. He sends the supervisors a list of the previous year's deliverables.

`TI_Meetings` Teams meet in person during class time.

`TI_OpenQuestion` Open question: how involved the platform will be in the capstone process.

CS Instructor (CI) Professor who runs the Computer Science capstone course.

`CI_Projects` Projects from other professors are collected through the form that the SE instructor sends out. Most communication about projects happens in August. He would like to widen who receives the form for external projects.

`CI_ProjectStatus` Wants project state kept up to date automatically, since supervisors currently have to report it. A project accepted by a team should become unavailable.

`CI_Approval` Listed projects are accepted automatically since he has already seen them. More than half of teams define their own project, which are handled through a discussion board; a board of proposals he can approve would be less disorganized.

`CI_TeamFormation` Students struggle with team formation, which currently relies on talking to friends. Features that help students communicate would help.

`CI_Config` Does not use GitLab; students keep GitHub repositories and share them with him. Assignment submission should stay where it is. Instructors' and TAs' access should be configurable.

`CI_NoExtraWork` Does not want engagement past the selection stage, and expects supervisors to contact him if there is a problem. Features should be optional and intuitive and should not create extra work.

`CI_ExternalData` Open question: whether data can stay internal when external supervisors are involved, and whether internal hosting would prevent supervisors from accessing it.

CI8Requested a way to book twice-per-term meetings with teams and to remind teams of missed meetings, supervisor-instructor communication per project, and record keeping between students and supervisors or instructors. Meeting booking is outside the scope.

13.2 Appendix B: Development Plan

14 Development Plan

Requirements are grouped into phases. Phase 1 creates the smallest end-to-end path through the lifecycle (see Figure 2), so that a project can be proposed, approved, listed, selected, and finally approved for a team. Priority within a phase follows the instructor's biggest pain point(repetitive review of team submissions) and then the dependencies between features.

14.1 Phase 1: Core Workflow (implemented in the course)

- Importing course enrollment from Avenue to Learn, with file upload as an alternative, and a list of enrolled students who are not on a team (UC_ImportEnrollment; SR_Enrollment). Team validation (UC_ValidateTeams) needs to know who is enrolled, so this comes first.
- External supervisor sign-in with an unverified state, verification by approval of the supervisor's own proposal, and scoped visibility (SR_ExternalSupervisorAccess). This is needed before any real sponsor uses the platform, so it is not deferred.
- Supervisor proposal submission with a structured template, and instructor approval before a project is listed (UC_SubmitProposal, UC_ReviewProposal; SR_ProposalTemplate, SR_OptionalFields, SR_HumanApproval).
- Browsing projects with a visible status, and team selection (UC_BrowseProjects, UC_SelectProject; SR_BrowseProjects, SR_ProjectStatus).
- Team project submission with automated checks, and instructor and TA review with iterative feedback (UC_ApproveTeamProject; SR_TeamSubmissionChecks, SR_ReviewDialogue, SR_TAApproval). This is the highest-value feature for the instructor, so it is part of the first working version.
- Validated team composition export (UC_ValidateTeams; SR_TeamValidity, SR_ReduceAdmin).

14.2 Phase 2: Communication and Visibility (implemented in the course)

- Email notifications for status changes and tasks, and a checklist of required actions (UC_NotifyChecklist; SR_StatusNotify, SR_PushNotify, SR_Checklist).
- Team profiles visible to supervisors, and team contact details (UC_TeamProfile, UC_ViewTeam; SR_StudentProfiles, SR_TeamContacts).
- Course schedule and timelines (UC_ViewTimelines; SR_CrossCourseTimelines).
- Identifying unselected projects (UC_FollowUpUnselected; SR_UnselectedProjects).

14.3 Phase 3: Extended Features (implemented if time allows)

- Student-initiated project proposals (UC_ProposeOwnProject; SR_StudentProposals). Phase 1 already provides most of the approval flow, so this mainly adds a proposal path, a student-proposed project list that supervisors can browse, and an optional field for naming a supervisor.
- Guidelines, FAQ, examples of past projects, and early access to course information (SR_ProposalGuidance, SR_CourseInfoEarly). This is mostly content, so it can be

added without changing the core system.

- Supervisor capacity limits (a count of teams per supervisor) and related checks.

14.4 Future Work (not implemented, or faked for the demo)

- (SR_SinglePlace) a single place for all capstone coordination, is addressed in part by the above, but full replacement of external tools (e.g., email, calendar, meeting tools) is future work.
- Meeting notes, searchable communication history, and suggested meeting schedules are not in the current requirements. They would be added after the core system is stable.
- Matching teams to projects by skills and interests is future work. It is optional and the instructor wants it to be switchable.
- Support for other capstone courses or universities (modularity) is a design goal, but a second deployment is future work.
- Automatic deletion of student data at the end of a course cycle and deletion on request (DPR_EndOfTermStudentDataDeletion, DPR_UserManagedContentDeletion) are not part of the demo, which uses seeded data.

14.5 Faked or Simplified for the Course

- Email delivery may use a test mailbox or log messages instead of reaching real inboxes during demos.
- Authentication may use a simplified sign-in for the demo, with real integration with university accounts as future work.
- Data may be seeded with example projects, teams, and supervisors instead of imported from the existing forms and spreadsheets.

14.6 Appendix C: Values of Auxiliary Constants

The following auxiliary constants show the values of symbolic parameters introduced within the NFRs Section 8.

Constant	Value	Description	NFR Reference
Maintenance Effort	20%	Maintenance effort as a fraction of total development effort.	NFR_ChangeEffort

Usability Success Percentage	75%	Percentage of first-time test participants who successfully complete their primary tasks.	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted
Usability Test Time	5 minutes	Time for first-time users to complete their primary tasks.	NFR_UsabilityFirstSuccess, NFR_LearnUnassisted
Deploy Time	30 minutes	Deployment time by IT professional with a deployment guide and no assistance.	NFR_DeployUnassisted
Uptime Percent	99%	Minimum required system uptime.	NFR_Uptime
Unscheduled Downtime	$\sim$ 1.7 hours	Maximum unscheduled hours of downtime per week.	NFR_Uptime
Tail Percentile	95th	Percentile used to evaluate response latency.	NFR_PerfTail
Tail Latency	5.0 seconds	Maximum response latency at the specified percentile.	NFR_PerfTail, NFR_PeakUsers
Concurrent Users	400	Peak number of concurrent users simulated during performance testing.	NFR_PerfTail, NFR_PeakUsers
User Think Time	10 seconds	Pause between actions for each simulated user during performance testing.	NFR_PerfTail
Recovery Point Window	24 hours	Maximum age of the restored state relative to the incident, except when no recovery point within the window is available (see NFR_Recoverable).	NFR_Recoverable

Backup Interval	24 hours	Maximum time between backups. Must not exceed the recovery point window.	DPR_DailyDataBackups, NFR_Recoverable
Health Check Interval	1 minute	Time between availability health checks.	NFR_Uptime
Availability Window	30-day	Period over which uptime is measured.	NFR_Uptime

Table 9: Auxiliary Constants

14.7 Appendix D: Generative AI Disclosure

Used Claude Sonnet 5.5 for the following,

- Came up with potential persona names.
- Help draft possible scenarios where a stakeholder could run into issues using the app.
- Used it to check both documents against the checklist and rubric and asked it to flag anything that was missing and any grammar issues

Used ChatGPT 6 for the following,

- Functional Requirements (Section 6): To help brainstorm potential requirements that I would have missed.
- Data Requirements (Section 7): Mostly structuring and reviewing to identify inconsistencies.
- Traceability Matrix (Section 10): Asked it about requirement overlaps I was unsure about before finalizing them.

14.8 Appendix E: Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not

based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Waqar:

1. What went well while writing this deliverable? During this deliverable, distributing work was very straightforward as there was a lot of work, but it was separable by section. It was all work we have seen before in prior courses with documentation, so it was very intuitive to write and come up with solutions/answers.

2. What pain points did you experience during this deliverable, and how did you resolve them? During this deliverable, everyone was very busy because we had midterms and a lot of assignments due alongside this report. Sometimes it also became difficult to stay updated with everyone's parts. However, because we planned out our schedule well in advance, we were able to finish everything on time while also completing our other work. Everyone also posted updates in our team chat when they got the chance, so we all stayed in the loop.

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)? All of our stakeholder requirements came from conversations with stakeholders as well as with each other (we are students as well). We interviewed a student who had been through project selection, an external supervisor, an internal supervisor (a professor), and the instructor who runs the course. Each requirement is tagged with the stakeholder(s) who raised it, and the interview notes are in Appendix A so each one can be traced back to what was actually said. For example, the structured proposal template came from both supervisors and the student, the project status visibility came from the student, external supervisor, and instructor, and the instructor's preference for human-only approval decisions shaped our approval requirements. Our personas and use cases were also built from the same interview notes. The only parts that came from our own judgment rather than a direct request were implementation-level details.

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project. Definitely SFWRENG 4HC3 which is user interfaces will be very useful later on as we start to actually design the idea for the UI/UX. This will be useful not just the implementation but also in the design development using tools like figma. Also, this course helps consider stakeholders which is exactly what we are doing right now so it's helping with not just interview development but also coming up with stakeholders. Furthermore, much of our stretch goals rely on AI, so SFWRENG 4AL3 will also be useful as it will help to develop the implementation behind those goals, like group suggestions based on other data like skills we collect of

students.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member. For more domain specific knowledge, we need to understand how the capstone courses run: proposal intake, approval criteria, timelines across the SE, CS, and Tron courses, and the role of supervisors, TAs, and the instructor. We will get this through further conversations with the instructor and supervisors. Another point to consider is, Our stakeholders already use GitLab/GitHub merge requests for approval and email for communication. We need to learn how to integrate with or replace these (e.g., GitLab's API, email notification services) without making the workflow worse.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice? For the first domain I mentioned; First we can interview other instructors (Tron, CS) and supervisors and ask them to walk through a full term. Another method would be read past course outlines, the proposal forms, and the existing GitLab repository and scripts to really understand the process on a more granular level. I will continue to pursue this.

   For the second bit of knowledge I mentioned, the first thing I can do is read the official API documentation and build a small prototype to see if I can integrate with the current process. The second thing I can do is follow tutorials or examples of similar integrations to see how manageable it is. I can pursue this knowledge as well.

Ayush:

1. What went well while writing this deliverable? All the work in this section was something we had seen before in previous courses, especially in 3RA3. This made it easy for us to know exactly what was expected for each section. We had a lot of experience coming up with functional and non function requirements, use cases and scenarios. We even have experience doing requirements traceability matrices. This made it quite intuitive to write this document.

2. What pain points did you experience during this deliverable, and how did you resolve them? Everyone was quite busy because we had many assignments and midterms due at the same time as this deliverable, which made it hard to prioritize and work on the SRS. Also, many of the parts of this document build on top of one another and they all must stay consistent, which made it hard to parallelize and split up the work. We often had to wait for one part to be done before the next can be started. For example,

the introduction and scope sections must be well defined before we can start writing the use cases and the scenarios, and those blocked the functional and non-functional requirements.

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)? All of our requirements came from speaking with stakeholders and people who have dealt with this problem before (including ourselves as students). We have spoken with a few students, Dr. Smith, an external industry supervisor, as well as a McMaster faculty supervisor. Our discussions with these stakeholders has guided all of our requirements and all of our decisions. Many of the requirements about modularity, removing repetitive approvals and knowing which projects have not been taken came directly from Dr. Smith.

   Requirements regarding a template for student proposed projects, visibility into project status and help with team formation came diectly from our conversations with a student.

   Requirements about having access to more information about the course processes, expectations and timelines, and knowing more about the students, and the requirements about notifications came directly from talking to industry supervisors.

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

   As mentioned above, SFWRENG 3RA3, our requirements course, has been the most helpful in completing this SRS as well as thinking about the bigger picture of the project. What we learned in that course has helped us focus on users, making sure that we are building something that actually solves problems that they have.

   Another course that has been helpful for us is SFWRENG 4HC3. In this course, we learned about to interview stakeholders to get the most value out of interviews. We learned about the different types of interviews and we actually conducted many semi-structured interviews for this document. Additionally, since our project will most likely consist of a web applications, what we learn in this course about interface design will be quite beneficial to this project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

   In terms of domain specific knowledge, we will need to acquire knowledge about how different capstone courses work and all the processes involved. There is a lot that goes on behind the scenes of a successful capstone course and every course does something

different. Given that our platform should be extensible to other capstone courses at McMaster and even other universities, we need to know more about how these different capstone courses operate. We need to know how they come up with project ideas, how do they do team formation, how do the instructors approve projects. Knowing these things will allow us to make our design as abstract as possible so it can be extended to different courses.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   To gain more domain specific knowledge, we will interview the relevant stakeholders, including the instructors and students of these courses. For example, in CAS, we still need to interview Dr. Moradi and Dr. Wassyng about their capstone process. Another way to get more domain knowledge is to research online how other instutitions do things like project management and effective team formation.

Ahmed:

1. What went well while writing this deliverable?

   All group members had lots of experience writing SRS documents through courses like 3RA3 and 3A04. No stone was left unturned, and the document was very comprehensive covering stakeholder, functional, non-functional, and data requirements. Speaking with stakeholders was very insightful, inspiring most of the requirements written.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   The way the IEEE template covered requirements in a different manner than the standard Volare-style template. Some adjustments were needed to adopt the previous way of writing non-functional requirements to the IEEE style. Swapping the specifically the NFR section template from IEEE to Volare was considered, but it was determined to have likely introduced many duplicate requirements.

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

   The majority of requirements were inspired by stakeholder interviews. Some were added as a result of other design processes (such as the Hazard Analysis) and from industry standard practices from experiences in the workforce.

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

   SFWRENG 3RA3 is the primary course which taught the necessary skills for writing a comprehensive SRS. SFWRENG 3A04 provided additional practice which reinforced

good habits, such as, ensuring NFRs are measurable and centering functional requirements around use-cases. SFWRENG 3DB3 will provide foundational database skills, essential to a data-centered platform like this project is.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

   Domain knowledge in how capstone courses are run will be an ongoing pursuit for the duration of the project, ensuring a refined final product. It's foundational to the project, and missing the mark will result in a product that does not get adopted for the target use cases.

   Additionally, code styles have the tendency to clash, which means each team member will need to learn the proper conventions for documentation, technology features used/avoided, and drawing interface boundaries, which are different to each part of the technology stack.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   Domain knowledge on capstone courses will be expanded on by meeting with McMaster capstone instructors throughout the course of developement. Another approach would be to get information from regulatory and institutional bodies which are responsible for standardizing capstone courses. Meeting with McMaster capstone instructors provides first-hand insight into the process which a set of standards and regulations cannot provide, and is the approach that will be taken.

   One way to synchronize code styles is to specify it up-front using descriptive language and a set of rules. Another way would be to select the most experienced individual for each technology and have them be responsible for reviewing all code which uses that technology and only approve changes when they meet the standard. The team will move forward with the latter approach, with technology leader selection being something that emerges rather than being decided on prematurely.

Andrew:

1. What went well while writing this deliverable? What went well in this deliverable was the fact that we had done this before in many software courses so it was not new to us. At the same time, we held very thorough interviews before the SRS which made writing this document significantly easier.

2. What pain points did you experience during this deliverable, and how did you resolve them? During the deliverable, a pain point was aligning with team members since we were very busy during the milestone. As a result of the misalignment in schedules, we had to wait for each other to finish pre-requisites to other sections before we could start working on them. We resolved this through excellent communication on Teams and planning how we can finish the deliverables. Another pain point was formalizing our requirements. It was difficult to turn our stakeholders' many requests into a formal SRS. This was resolved by categorizing requests and finding overlaps between requests to ensure there were no redundancies and we could obtain full coverage on what was asked.

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

   As per the traceability matrix, nearly all our functional requirements can be traced back to a request from our stakeholders during our interviews with them. These stakeholders included a capstone instructor, a student, and an external supervisor so far. Since we are designing the application for the target audience of the system, the requirements were centered around what these groups are looking for to make the capstone experience smoother. Every functional requirement formally articulated what the potential users were looking for. A requirement such as FR28 relating to McMaster SSO or external auth is one of the exceptions but is valid since it indirectly serves stakeholders by allowing their information to remain secure.

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project. 3RA3, 4HC3, and 4A03 were all very useful courses for this type of document. In each of these courses, we were also working on a big project that required us to formalize our requirements into a document while keeping in mind system design. We also evaluated the needs of our stakeholders giving us lots of practice before the capstone's SRS.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member. For domain specific knowledge, we need to understand in detail how each course works and how to create an abstract solution that can integrate all three course processes. We will still have to obtain knowledge on how the remaining instructors for CS and TRON run their courses and how it may conflict with what we obtained from Dr. Smith. We will need a better understanding of how to create scalable usable systems that are compatible with McMaster's current infrastructure particularly with respect to the capstone course. This includes where we will deploy and what storage solutions we will use. We will try

to figure out the algorithms and APIs that will work for our solution including how to integrate with Gitlab. Finally, we need to understand how we can present our results, work, and demonstration.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice? Course Process (Ayush leads while remaining in attendance): Interview CS/Tron Prof, Finding similarities and conflicts between stakeholders. APIs (Waqar while rest of team helps): Read documentation, Follow tutorials. Presentation (Ahmed taking lead while rest of team helps): Research on how capstones presented, determining how our final product should look from UI/UX perspective. Infrastructure Compatibility and Deployment (Andrew while rest of team supports): Further discussions with professors will help with this, Researching deployment practices and scalable systems.

References

ISO/IEC/IEEE. Systems and software engineering — life cycle processes — requirements engineering. Technical Report ISO/IEC/IEEE 29148:2018, 2018.

ISO/IEC/IEEE. Systems and software engineering — system life cycle processes. ISO/IEC/IEEE Standard 15288:2023, IEEE, 2023.